

MH1812 Discrete Mathematics
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumsclaw/ntu-cs-textbooks

Contents

1	Logic	1
1.1	Propositions and Connectives	1
1.2	Truth Tables as Exhaustive Checklists	2
1.3	Predicates and Quantifiers	3
1.3.1	Normal Forms	4
1.4	Worked Example: From English to Formula and Back	4
1.5	Chapter Notes	5
1.6	Exercises	5
2	Sets	6
2.1	Sets and Membership	6
2.2	Power Sets and Cartesian Products	7
2.3	Relations	8
2.4	Worked Example: Counting with Sets	9
2.5	Chapter Notes	10
2.6	Exercises	10
3	Functions	11
3.1	Functions as Deterministic Arrows	11
3.2	Inverses and Composition	12
3.3	Cardinality: Comparing Sizes via Bijections	13
3.4	Worked Example: Bijections in Action	15
3.5	Chapter Notes	15
3.6	Exercises	15

4	Proof	16
4.1	Direct Proof	16
4.2	Contrapositive	17
4.3	Contradiction	17
4.4	Induction and Strong Induction	18
4.5	Worked Example: A Complete Induction Proof Dissected	20
4.6	Chapter Notes	20
4.7	Exercises	20
5	Counting	22
5.1	Product and Sum Rules	22
5.2	Permutations and Combinations	23
5.3	Pigeonhole Principle	24
5.4	Worked Example: Two Ways to Count	26
5.5	Chapter Notes	26
5.6	Exercises	26
6	Recurrences	27
6.1	What Is a Recurrence?	27
6.2	Solving by Iteration (Unrolling)	28
6.3	Linear Homogeneous Recurrences	28
6.4	Induction Proofs for Closed Forms	29
6.5	Worked Example: Solving a Non-Homogeneous Recurrence End-to-End	31
6.6	Chapter Notes	31
6.7	Exercises	31
7	Graphs	32
7.1	Graphs and Degrees	32
7.2	Isomorphism	33
7.3	Euler Trails and Circuits	33
7.4	Hamilton Cycles and Trees	34

7.5	Worked Example: Euler vs. Hamilton on the Same Graph	35
7.6	Chapter Notes	36
7.7	Exercises	36
8	Number Theory	37
8.1	Divisibility and the Division Algorithm	37
8.2	GCD and Euclid	38
8.3	Modular Arithmetic	39
8.4	RSA Intuition	39
8.5	Worked Example: A Full Modular Calculation	41
8.6	Chapter Notes	41
8.7	Exercises	41

Chapter 1

Logic: Propositions, Truth Tables, Predicates and Quantifiers

“Logic is the anatomy of thought.” — John Locke. Every program branch, loop condition, and database query is a logical formula. We build logic from everyday language to formal syntax.

From zero: no prior logic assumed. We start with ordinary sentences (“It is raining”), turn them into symbols, build truth tables as exhaustive checklists, then lift to predicates with variables and quantifiers. Pictures precede formalism.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish propositions from non-propositions and translate English to propositional formulae;
- (L2) build and use truth tables to test tautology, contradiction, and logical equivalence;
- (L3) apply De Morgan, distributive, and implication equivalences in proofs and simplifications;
- (L4) formalise statements with predicates and quantifiers $\forall, \exists$;
- (L5) negate quantified statements correctly and explain quantifier-order dependence.

1.1 Propositions and Connectives

A *proposition* is a declarative sentence that is either true (T) or false (F) — no maybe, no command, no question. “ $2 + 2 = 4$ ” is T; “ $2 + 2 = 5$ ” is F; “Are you there?” is not a proposition; “ $x + 1 = 3$ ” is not until x is bound.

We combine propositions with *connectives*. Intuition: think of switches.

Symbol	Name	Intuition
$\neg p$	not p	flip the switch
$p \wedge q$	p and q	both lights on
$p \vee q$	p or q	at least one on (inclusive)
$p \rightarrow q$	if p then q	promise: p true forces q true
$p \leftrightarrow q$	p iff q	same truth value

series: both closed

parallel: one closed

only F when $p=T, q=F$

$\rightarrow \neg p \vee q$

Memorise: $p \rightarrow q$ false exactly when $p=T, q=F$

Figure 1.1: Intuition for connectives: series (AND), parallel (OR), and the promise reading of implication.

Definition 1.1 (Well-formed formula). Atomic propositions $p, q, r, \dots$ are formulae; if F, G are formulae so are $\neg F$, $(F \wedge G)$, $(F \vee G)$, $(F \rightarrow G)$, $(F \leftrightarrow G)$.

1.2 Truth Tables as Exhaustive Checklists

A truth table lists every assignment to the variables (2^n rows for n variables) and computes stepwise. This is brute-force semantics — slow but absolute.

Truth table for $(p \rightarrow q) \leftrightarrow (\neg p \vee q)$ is a tautology

p	q	$p \rightarrow q$	$\neg p \vee q$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

columns match on every row $\Rightarrow$ tautology

De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$

$\neg(p \wedge q)$	$\neg p \vee \neg q$
T	T
T	T
T	T
T	T

columns identical

Figure 1.2: Truth tables: structure and the two canonical equivalences every CS student must know.

Definition 1.2 (Tautology, contradiction, contingency). A formula true on every row is a *tautology* ($\top$); false on every row a *contradiction* ($\perp$); otherwise *contingent*.

Key equivalences (verify by table):

$$\begin{aligned}
 p \rightarrow q &\equiv \neg p \vee q \equiv \neg q \rightarrow \neg p && \text{(implication / contrapositive)} \\
 \neg(p \wedge q) &\equiv \neg p \vee \neg q, \quad \neg(p \vee q) \equiv \neg p \wedge \neg q && \text{(De Morgan)} \\
 p \wedge (q \vee r) &\equiv (p \wedge q) \vee (p \wedge r) && \text{(distributivity)} \\
 p \leftrightarrow q &\equiv (p \rightarrow q) \wedge (q \rightarrow p).
 \end{aligned}$$

Example 1.1 (Implication is not causation). “If it rains then the road is wet” ($r \rightarrow w$) is false only when it rains and the road stays dry. Wet road without rain does not falsify it. This matches $\neg r \vee w$.

1.3 Predicates and Quantifiers

A *predicate* $P(x)$ is a sentence with a variable: truth depends on x . Domain D matters. $P(x) : “x > 3”$ over $D = \mathbb{N}$ is T for 5, F for 2.

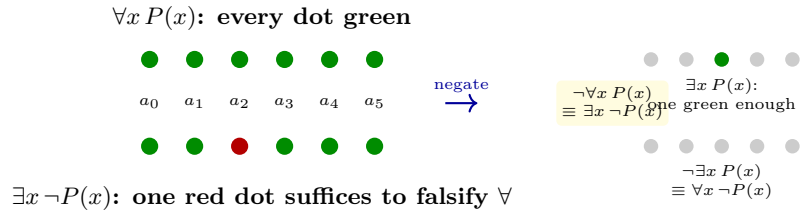


Figure 1.3: Quantifiers as dot-colouring. Negation swaps $\forall \leftrightarrow \exists$ and flips the predicate — the most common bug.

Definition 1.3 (Quantifiers). $\forall x \in D, P(x)$ means $P(x)$ holds for every $x \in D$. $\exists x \in D, P(x)$ means for at least one $x \in D$. Quantified formulae are propositions.

Negation swaps quantifiers:

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

Iterate for blocks: $\neg \forall x \exists y P(x, y) \equiv \exists x \forall y \neg P(x, y)$.

Example 1.2 (Order matters). Over $\mathbb{R}$, $\forall x \exists y (x + y = 0)$ is true (take $y = -x$), but $\exists y \forall x (x + y = 0)$ is false (no single y works for all x). Swapping quantifiers is not free.

Nested quantifiers model real CS specs: “Every input has some output such that ...” is $\forall \text{input} \exists \text{output } P$.

```

1 # Checking tautology by brute force (2^n rows) -- mirrors truth tables
2 import itertools
3 def is_tautology(n, f):
4     for vals in itertools.product([False, True], repeat=n):
5         if not f(*vals):
6             return False, vals
7     return True, None
8
9 # Example: (p -> q) <-> (~p or q) is a tautology
10 print(is_tautology(2, lambda p,q: ( (not p or q) == (not p or q) )))

```

Remark 1.1 (Translating English). “Every student who owns a laptop passed” = $\forall s (\text{Owns}(s) \rightarrow \text{Passed}(s))$, not $\forall s (\text{Owns}(s) \wedge \text{Passed}(s))$. The $\rightarrow$ restricts the claim to laptop-owners.

Example 1.3 (De Morgan in code). In Python/Java, $!(a \ \&\& \ b)$ should be rewritten as $!a \ || \ !b$. Many bug fixes are just De Morgan. Truth tables guarantee correctness of such rewrites.

Lemma 1.1 (Contrapositive equivalence). $p \rightarrow q \equiv \neg q \rightarrow \neg p$. Proof by table or via $\neg p \vee q$: $\neg q \rightarrow \neg p \equiv \neg(\neg q) \vee \neg p \equiv q \vee \neg p \equiv \neg p \vee q$.

1.3.1 Normal Forms

Every formula has a *conjunctive normal form* (CNF): an AND of OR-clauses, e.g. $(p \vee \neg q) \wedge (r \vee q)$. CNF underlies SAT solvers that power hardware verification. Dually, *disjunctive normal form* (DNF) is an OR of AND-terms. Conversion uses distributive and De Morgan laws repeatedly.

Example 1.4 (Tautology via equivalences). $(p \rightarrow q) \wedge (q \rightarrow r) \rightarrow (p \rightarrow r)$ is a tautology (hypothetical syllogism). Proof: chain $\neg p \vee q$, $\neg q \vee r$ resolves to $\neg p \vee r$.

Remark 1.2 (Biconditional as two directions). $p \leftrightarrow q$ is often proved by two subproofs $p \rightarrow q$ and $q \rightarrow p$. In code this is `assert (p==q)` checked both ways.

Theorem 1.1 (Exportation). $(p \wedge q) \rightarrow r \equiv p \rightarrow (q \rightarrow r)$. Currying in functional programming is this equivalence.

Intuition: a proposition is like a light switch, either on or off, never in-between. In code, an if-condition is precisely a proposition evaluated at run time. Truth tables are exhaustive: with n variables we must check all 2^n assignments. Equivalence $p \rightarrow q \equiv \neg p \vee q$ explains why `if (p) q` can be rewritten as `if (!p || q)`. De Morgan is the formal justification for pushing negations inside brackets. Quantifiers turn predicates with holes into full propositions by binding the holes. The order $\forall x \exists y$ versus $\exists y \forall x$ is as different as every person has a mother versus there is a mother of everyone. Translating word problems to symbols is half the battle in discrete mathematics. A common error is to write $\forall x(P(x) \wedge Q(x))$ when $\forall x(P(x) \rightarrow Q(x))$ was intended. Practice negating statements aloud to internalise the swap-and-flip rule. Intuition: a proposition is like a light switch, either on or off, never in-between. In code, an if-condition is precisely a proposition evaluated at run time. Truth tables are exhaustive: with n variables we must check all 2^n assignments. Equivalence $p \rightarrow q \equiv \neg p \vee q$ explains why `if (p) q` can be rewritten as `if (!p || q)`. De Morgan is the formal justification for pushing negations inside brackets. Quantifiers turn predicates with holes into full propositions by binding the holes. The order $\forall x \exists y$ versus $\exists y \forall x$ is as different as every person has a mother versus there is a mother of everyone. Translating word problems to symbols is half the battle in discrete mathematics. A common error is to write $\forall x(P(x) \wedge Q(x))$ when $\forall x(P(x) \rightarrow Q(x))$ was intended. Practice negating statements aloud to internalise the swap-and-flip rule.

1.4 Worked Example: From English to Formula and Back

Consider “If the server is up then the database is reachable, and the server is up, therefore the database is reachable.” Let p : server up, q : database reachable. Premises: $p \rightarrow q$, p ; conclusion q (modus ponens). Truth table: rows where $p \rightarrow q$ and p are T only when $p = \text{T}$, $q = \text{T}$, so q holds. This is why unit tests encode implications as `assert (not p) or q`.

Another example: “You can have dessert iff you finished vegetables.” This is $d \leftrightarrow v$, equivalent to $(d \rightarrow v) \wedge (v \rightarrow d)$. Negation $\neg(d \leftrightarrow v)$ is $(d \wedge \neg v) \vee (\neg d \wedge v)$ — exactly exclusive-or.

Translating quantifiers: “Every even integer is the sum of two primes” (Goldbach, unknown) is $\forall n \in 2\mathbb{Z}^+, (n > 2 \rightarrow \exists p, q \text{ prime } n = p + q)$. Its negation $\exists n (n \text{ even } > 2 \wedge \forall p, q \text{ prime } n \neq p + q)$ shows the logical complexity.

1.5 Chapter Notes

Logic originates with Aristotle and was formalised by Frege, Russell and Whitehead. Truth tables are due to Wittgenstein/Post. For CS, Rosen Ch. 1 and Epp Ch. 2 cover these equivalences; TLA⁺ and SQL WHERE clauses are applied predicate logic.

1.6 Exercises

- E1.1 **Propositions.** Which are propositions? (a) “ $3 < 5$ ”, (b) “Close the door”, (c) “ $x^2 = 4$ ”, (d) “There exists an even prime”. For propositions give truth value; for non-propositions explain why.
- E1.2 **Truth tables.** Use a table to decide tautology/contradiction/contingency for (a) $(p \wedge q) \rightarrow p$, (b) $(p \rightarrow q) \wedge (q \rightarrow p) \leftrightarrow (p \leftrightarrow q)$, (c) $(p \vee q) \wedge \neg p \wedge \neg q$.
- E1.3 **Equivalences.** Without a full table, use known equivalences to show $\neg(p \rightarrow q) \equiv p \wedge \neg q$ and simplify $\neg(\neg p \wedge q) \vee p$ to a tautology. Name each step.
- E1.4 **Quantifier translation.** Domain S = students, C = courses. Translate (a) “Every student took some course”, (b) “Some course is taken by every student”, (c) negate (a) in plain English without “not every” or “it is not the case”.
- E1.5 **Quantifier order.** Give a domain D and predicate $P(x, y)$ where $\exists x \forall y P(x, y)$ is true but $\forall y \exists x P(x, y)$ is also true yet meanings differ; then give one where the second is true and the first false. Explain.

Chapter 2

Sets, Power Sets, Venn Diagrams and Relations

“A set is a Many that allows itself to be thought of as a One.” — Georg Cantor. Databases, types, and state spaces are all sets.

From zero: sets from dots. Picture a bag of dots; a set is just which dots are in the bag. We formalise membership, build operations (union, intersection, complement), count subsets (power set), draw Venn regions, and then generalise to relations as sets of pairs.

Learning Outcomes

After this chapter you will be able to:

- (L1) use set notation ($\in, \subseteq, \subsetneq, |S|, \emptyset, \mathcal{P}(S), A \times B$) fluently;
- (L2) compute unions, intersections, complements, differences and power sets;
- (L3) count subsets and interpret $|\mathcal{P}(S)| = 2^{|S|}$;
- (L4) read and shade Venn diagrams for arbitrary set expressions;
- (L5) define relations, their properties, closures, and equivalence/partial orders.

2.1 Sets and Membership

A *set* is an unordered collection of distinct objects. $x \in S$ means x is a member; $x \notin S$ otherwise. $S \subseteq T$ if every $x \in S$ also $x \in T$. $S = T$ iff $S \subseteq T$ and $T \subseteq S$. $S \subsetneq T$ if $S \subseteq T$ and $S \neq T$. $\emptyset$ has no members; $|\emptyset| = 0$. Universe U is the ambient set for complements.

Common universes: $\mathbb{N} = \{0, 1, 2, \dots\}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{Z}^+$, $[n] = \{1, \dots, n\}$. Interval $[a, b]$, (a, b) have usual meaning over $\mathbb{R}$.

Operations (picture as Venn shading):

$$\begin{aligned} A \cup B &= \{x : x \in A \vee x \in B\}, \\ A \cap B &= \{x : x \in A \wedge x \in B\}, \\ A \setminus B &= \{x : x \in A \wedge x \notin B\}, \quad \bar{A} = U \setminus A, \\ A \oplus B &= (A \setminus B) \cup (B \setminus A) \quad (\text{symmetric difference}). \end{aligned}$$

Laws mirror logic: De Morgan $\overline{A \cup B} = \bar{A} \cap \bar{B}$, distributivity, etc., because $x \in A \cup B \equiv x \in A \vee x \in B$.

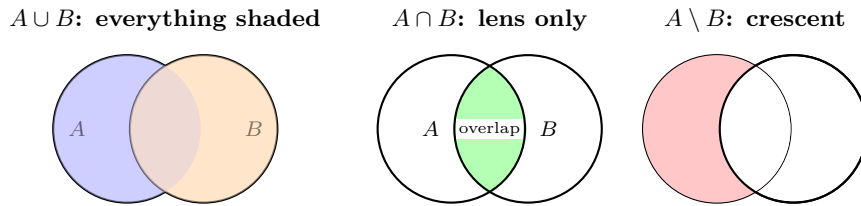


Figure 2.1: Three Venn shadings that translate directly to $\vee$, $\wedge$, $\wedge \neg$. Colour = membership.

Example 2.1 (Set laws by element argument). $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$. Proof: $x \in \text{LHS} \iff x \in A \wedge (x \in B \vee x \in C) \iff (x \in A \wedge x \in B) \vee (x \in A \wedge x \in C) \iff x \in \text{RHS}$.

2.2 Power Sets and Cartesian Products

The *power set* $\mathcal{P}(S) = \{T : T \subseteq S\}$ collects all subsets. If $S = \{a, b, c\}$ then $\mathcal{P}(S)$ has $2^3 = 8$ members: $\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}$.

Theorem 2.1 ($|\mathcal{P}(S)| = 2^{|S|}$). For finite S with $|S| = n$, $|\mathcal{P}(S)| = 2^n$. Each subset corresponds to an n -bit string (choose in/out per element); 2 choices per position $\Rightarrow 2^n$ strings.

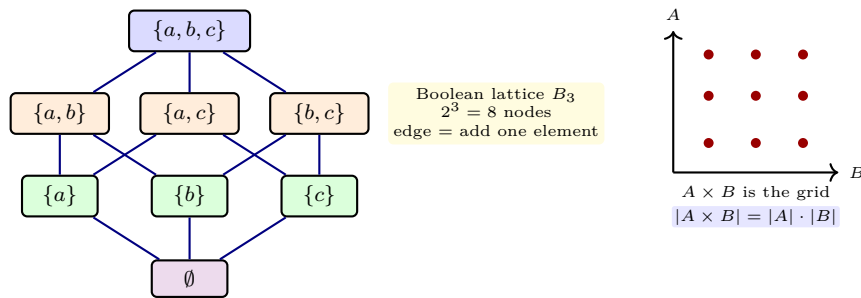


Figure 2.2: Left: power set as Boolean lattice (Hasse diagram). Right: Cartesian product as grid of ordered pairs.

The *Cartesian product* $A \times B = \{(a, b) : a \in A, b \in B\}$ has $|A \times B| = |A| \cdot |B|$ (product rule, Ch. 5). $A \times B \neq B \times A$ generally; order matters.

```

1 # Power set via bit masks -- matches |P(S)| = 2^n
2 def powerset(s):
3     s = list(s)
4     n = len(s)
5     return [s[j] for j in range(n) if (mask >> j) & 1 for mask in range(1 << n)]
6 print(powerset({'a', 'b', 'c'}))

```

2.3 Relations

A *relation* R from A to B is any $R \subseteq A \times B$. Write $a R b$ for $(a, b) \in R$. On a single set A , $R \subseteq A \times A$ is a relation on A .

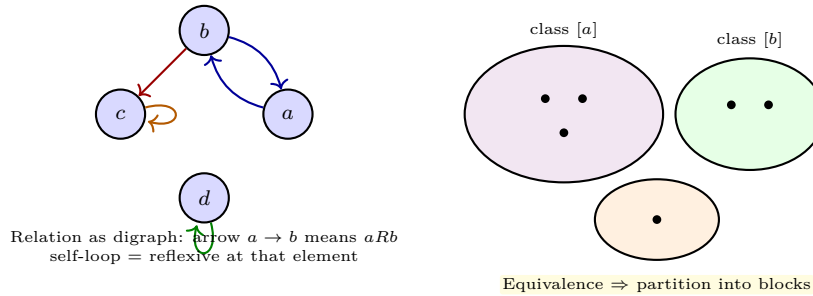


Figure 2.3: Left: digraph of a relation. Right: equivalence relation partitions the set into disjoint blocks.

Properties for R on A : *reflexive* ($\forall a \ a R a$), *symmetric* ($a R b \Rightarrow b R a$), *antisymmetric* ($a R b \wedge b R a \Rightarrow a = b$), *transitive* ($a R b \wedge b R c \Rightarrow a R c$).

Definition 2.1 (Equivalences and orders). R is an *equivalence* if reflexive, symmetric, transitive — it partitions A into *equivalence classes* $[a] = \{b : a R b\}$. R is a *partial order* if reflexive, antisymmetric, transitive; a *total order* if additionally comparable.

```

1  # Sets in Python directly mirror the maths
2  A = {1,2,3,4}; B = {3,4,5}
3  print(A | B, A & B, A - B, A ^ B)  # union, inter, diff, sym diff
4  # Relation as set of pairs; check reflexivity
5  def is_reflexive(R, A): return all((a,a) in R for a in A)
6  R = {(1,1), (2,2), (3,3), (1,2), (2,1)}
7  print(is_reflexive(R, {1,2,3}))

```

Example 2.2. “=” and “same birthday” are equivalences; “ $\leq$ ” on $\mathbb{Z}$ and “ $\subseteq$ ” on $\mathcal{P}(S)$ are partial orders (the latter not total for $|S| \geq 2$).

Closures: reflexive/symmetric/transitive closures add minimal pairs to achieve the property; transitive closure R^* is reachability in the digraph (Floyd–Warshall / BFS).

Remark 2.1 (Why $\emptyset \subseteq S$ always). $\emptyset \subseteq S$ vacuously: “ $\forall x(x \in \emptyset \rightarrow x \in S)$ ” has false antecedent for every x , so the implication is true on every row. Similarly $S \subseteq S$.

Theorem 2.2 (Inclusion–Exclusion for two sets). $|A \cup B| = |A| + |B| - |A \cap B|$. Venn lens is counted twice, subtract once.

Proof. Count $x \in A \cup B$: if x in exactly one, contributes 1; if in both, $1 + 1 - 1 = 1$. □

Example 2.3 (Set builder pitfalls). $\{x \in \mathbb{Z} : x^2 < 5\} = \{-2, -1, 0, 1, 2\}$. Always state the universe; $\{x : x^2 < 5\}$ over $\mathbb{R}$ is $(-\sqrt{5}, \sqrt{5})$.

Theorem 2.3 (Distributivity). $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ and dually $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$.

Remark 2.2 (Russell’s paradox sketch). The “set of all sets not containing themselves” cannot exist consistently — reason ZFC restricts comprehension. Not needed for finite CS sets, but warns against naive “set of all sets”.

Venn diagrams are not proofs, but they are excellent for conjecturing set identities before proving them algebraically. The empty set is a subset of every set, vacuously, which often serves as the base case of set inductions. Power sets grow exponentially: adding one element doubles the number of subsets. Bit-strings and subsets are the same idea in different costumes, linked by characteristic functions. Cartesian products model tables in relational databases: each row is an ordered pair. A relation can be stored as a Boolean matrix or an adjacency list, mirroring its digraph view. Equivalence classes partition a set like countries partition the globe, disjoint and covering. Partial orders model dependencies: $a \leq b$ means a must come before b . Closures add the minimal edges needed, like adding the missing transitive flights to connect all reachable cities. Inclusion–exclusion corrects overcounting, essential for counting unions without double-counting overlaps. Venn diagrams are not proofs, but they are excellent for conjecturing set identities before proving them algebraically. The empty set is a subset of every set, vacuously, which often serves as the base case of set inductions. Power sets grow exponentially: adding one element doubles the number of subsets. Bit-strings and subsets are the same idea in different costumes, linked by characteristic functions. Cartesian products model tables in relational databases: each row is an ordered pair. A relation can be stored as a Boolean matrix or an adjacency list, mirroring its digraph view. Equivalence classes partition a set like countries partition the globe, disjoint and covering. Partial orders model dependencies: $a \leq b$ means a must come before b . Closures add the minimal edges needed, like adding the missing transitive flights to connect all reachable cities. Inclusion–exclusion corrects overcounting, essential for counting unions without double-counting overlaps. Venn diagrams are not proofs, but they are excellent for conjecturing set identities before proving them algebraically. The empty set is a subset of every set, vacuously, which often serves as the base case of set inductions. Power sets grow exponentially: adding one element doubles the number of subsets. Bit-strings and subsets are the same idea in different costumes, linked by characteristic functions. Cartesian products model tables in relational databases: each row is an ordered pair. A relation can be stored as a Boolean matrix or an adjacency list, mirroring its digraph view. Equivalence classes partition a set like countries partition the globe, disjoint and covering. Partial orders model dependencies: $a \leq b$ means a must come before b . Closures add the minimal edges needed, like adding the missing transitive flights to connect all reachable cities. Inclusion–exclusion corrects overcounting, essential for counting unions without double-counting overlaps. Venn diagrams are not proofs, but they are excellent for conjecturing set identities before proving them algebraically. The empty set is a subset of every set, vacuously, which often serves as the base case of set inductions.

2.4 Worked Example: Counting with Sets

How many NTU students take MH1812 or SC2001? Let A = MH1812 cohort, B = SC2001 cohort. Inclusion–exclusion gives $|A \cup B| = |A| + |B| - |A \cap B|$. If $|A| = 300$, $|B| = 250$, $|A \cap B| = 80$, then $|A \cup B| = 470$. Without subtracting the overlap we double-count.

Power set example: a 3-bit binary string $b_2b_1b_0$ encodes a subset of $\{a, b, c\}$: $b_i = 1$ means include element i . Thus 101 encodes $\{a, c\}$. This bijection between $\{0, 1\}^3$ and $\mathcal{P}(S)$ proves $|\mathcal{P}(S)| = 2^3$ and generalises to 2^n ; it is also how a computer stores a subset as a bitmask.

Relation closure example: $R = \{(1, 2)\}$ on $\{1, 2, 3\}$ is not transitive, not symmetric. Its transitive closure adds nothing beyond R (no chain of length 2), but symmetric closure adds $(2, 1)$, and reflexive closure adds $(1, 1), (2, 2), (3, 3)$.

2.5 Chapter Notes

Cantor created set theory (1870s); Russell's paradox forced axiomatisation (ZFC). Power sets link to bit-strings and to complexity (2^n subsets). Venn diagrams (John Venn, 1880) remain a debugging tool for SQL and Boolean logic.

2.6 Exercises

- E2.1 **Operations.** Let $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5\}$. Compute $A \cup B$, $A \cap B$, $A \setminus B$, $\bar{A}$ with $U = [1..6]$, $A \oplus B$, $A \times B$, and $|\mathcal{P}(A)|$.
- E2.2 **Venn shading.** Shade (a) $(A \cap \bar{B}) \cup (B \cap \bar{A})$ (which equals $A \oplus B$), (b) $\overline{A \cup B} \cup (A \cap B)$. Verify De Morgan $\overline{A \cap B} = \bar{A} \cup \bar{B}$ by shading both sides.
- E2.3 **Power set proof.** Prove by induction or binomial sum that $\sum_{k=0}^n \binom{n}{k} = 2^n$ counts subsets by size. List $\mathcal{P}(\{1, 2, 3, 4\})$ grouped by cardinality k .
- E2.4 **Relations.** On $A = \{1, 2, 3\}$ let $R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$. Check each property and decide if R is equivalence or partial order. Find its equivalence classes or explain failure.
- E2.5 **Closures.** For $R = \{(1, 2), (2, 3)\}$ on $\{1, 2, 3, 4\}$ find reflexive, symmetric, and transitive closures. Draw digraphs before and after closure.

Chapter 3

Functions: Injections, Surjections, Bijections and Cardinality

“A function is a machine that turns inputs into outputs.” — yet not every machine is reversible, and not every output is hit. Injectivity, surjectivity and size make this precise.

From zero: arrows between bags. Picture two bags of dots (domain, codomain) with arrows $a \mapsto f(a)$. We classify when arrows never collide (injective), cover everything (surjective), or both (bijective), then use bijections to compare sizes — even for infinite sets.

Learning Outcomes

After this chapter you will be able to:

- (L1) define function, domain, codomain, image, preimage, composition;
- (L2) test injectivity, surjectivity, bijectivity with arrow and algebraic criteria;
- (L3) invert bijective functions and reason about composition;
- (L4) compare cardinalities via bijections and prove countability results;
- (L5) explain why $|\mathbb{N}| = |\mathbb{Z}| = |\mathbb{Q}| < |\mathbb{R}|$ (diagonal).

3.1 Functions as Deterministic Arrows

A *function* $f : A \rightarrow B$ assigns to each $a \in A$ exactly one $b = f(a) \in B$. A is the *domain*, B the *codomain*. The *image* (range) $f(A) = \{f(a) : a \in A\} \subseteq B$ may be smaller than B . The *preimage* $f^{-1}(T) = \{a : f(a) \in T\}$ lifts subsets backwards.

Think of a lookup table: one output per input, never zero or two. Vertical-line test is the picture: every a has exactly one outgoing arrow.

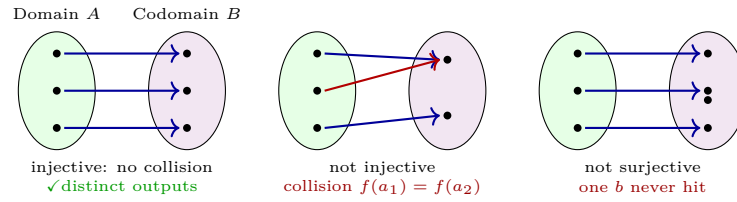


Figure 3.1: Arrow tests for injectivity and surjectivity. Bijective = both: perfect matching.

Definition 3.1 (Injective, surjective, bijective). $f : A \rightarrow B$ is *injective* (one-to-one) if $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$ (no two arrows share a target). *Surjective* (onto) if $\forall b \in B \exists a \in A f(a) = b$ (every b is hit). *Bijective* if both.

Example 3.1. $f : \mathbb{N} \rightarrow \mathbb{N}$, $f(n) = 2n$ is injective (if $2a = 2b$ then $a = b$) but not surjective (odd numbers missed). $g : \mathbb{R} \rightarrow \mathbb{R}$, $g(x) = x^3$ is bijective; $h : \mathbb{R} \rightarrow \mathbb{R}$, $h(x) = x^2$ is neither.

Composition $(g \circ f)(a) = g(f(a))$ is associative; identity $\text{id}_A(a) = a$ is bijective. If f, g injective then $g \circ f$ injective; similarly for surjective/bijective.

3.2 Inverses and Composition

$f : A \rightarrow B$ has an *inverse* $f^{-1} : B \rightarrow A$ iff f is bijective, with $f^{-1}(b) =$ the unique a with $f(a) = b$. Then $f^{-1} \circ f = \text{id}_A$ and $f \circ f^{-1} = \text{id}_B$.

Test: to show f bijective, exhibit an explicit inverse. For $f : \mathbb{Z} \rightarrow \mathbb{Z}$, $f(n) = n + 1$ has $f^{-1}(m) = m - 1$.

Lemma 3.1 (Composition preserves type). If $f : A \rightarrow B$ and $g : B \rightarrow C$ are bijections, so is $g \circ f$, with $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$ (socks-and-shoes).

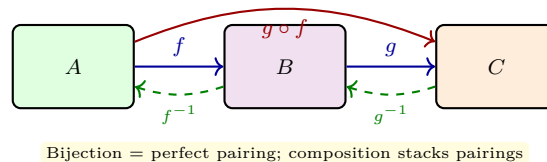


Figure 3.2: Composition and inverses: bijective arrows pair elements one-to-one.

```

1  # Checking injectivity/surjectivity by brute force on finite sets
2  def is_injective(f, A):
3      seen={}
4      for a in A:
5          b=f(a)
6          if b in seen: return False, (seen[b], a, b)
7          seen[b]=a
8      return True, None
9
10 def is_surjective(f, A, B): return all(any(f(a)==b for a in A) for b in B)
11 print(is_injective(lambda x: 2*x, {0,1,2,3}))
12 print(is_surjective(lambda x: 2*x, {0,1,2,3}, {0,1,2,3,4,5,6}))

```

3.3 Cardinality: Comparing Sizes via Bijections

We say $|A| = |B|$ iff a bijection $A \rightarrow B$ exists; $|A| \leq |B|$ iff an injection $A \rightarrow B$ exists. For finite sets this matches counting; for infinite sets it surprises.

$|\mathbb{N}| = |\mathbb{Z}|$: enumerate $0, 1, -1, 2, -2, \dots$ gives bijection $f : \mathbb{N} \rightarrow \mathbb{Z}$. $|\mathbb{N}| = |\mathbb{N} \times \mathbb{N}|$: Cantor pairing $(m, n) \mapsto 2^m 3^n$ injection and diagonal enumeration bijection; $|\mathbb{Q}| = |\mathbb{N}|$ via reduced fractions grid traversal.

But $|\mathbb{R}| > |\mathbb{N}|$. Cantor's diagonal: assume listing $r_1, r_2, \dots$ of $(0, 1)$ in decimal; build d whose i th digit differs from i th digit of r_i ; d not in listing — contradiction. Hence reals are *uncountable*.

$r_0 = 0.$	0	3	6	9	2
$r_1 = 0.$	1	4	7	0	3
$r_2 = 0.$	2	5	8	1	4
$r_3 = 0.$	3	6	9	2	5
$r_4 = 0.$	4	7	0	3	6

d differs from every r_i at position $i \Rightarrow$ not listed

$d = 0.d_0d_1d_2\dots$ with $d_i \neq$ diagonal digit i

Figure 3.3: Cantor diagonal: the red diagonal builds a real d absent from any enumeration, so $\mathbb{R}$ is uncountable.

Theorem 3.1 (Cantor–Bernstein–Schroeder, stated). If $|A| \leq |B|$ and $|B| \leq |A|$ then $|A| = |B|$.

Remark 3.1 (Pigeonhole preview). If $|A| > |B|$ no injection $A \rightarrow B$ exists (pigeonhole, Ch. 5). For finite A , $f : A \rightarrow A$ is injective iff surjective iff bijective.

Example 3.2 (Images and preimages). For $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x^2$, $f(\{-2, 2\}) = \{4\}$, $f^{-1}(\{4\}) = \{-2, 2\}$, $f^{-1}(\{-1\}) = \emptyset$. Images preserve unions; preimages preserve all Boolean operations.

Theorem 3.2 (Preimage preserves Boolean structure). For any $f : A \rightarrow B$ and $X, Y \subseteq B$, $f^{-1}(X \cup Y) = f^{-1}(X) \cup f^{-1}(Y)$, $f^{-1}(X \cap Y) = f^{-1}(X) \cap f^{-1}(Y)$, $f^{-1}(\bar{X}) = \overline{f^{-1}(X)}$.

Remark 3.2 (Why codomain matters). $f : \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto x^2$ is not surjective; $g : \mathbb{R} \rightarrow [0, \infty)$, $x \mapsto x^2$ is surjective. Same rule, different declared codomain — surjectivity is a claim about the declaration.

Remark 3.3 (Function vs. relation view). A function $f : A \rightarrow B$ is the relation $\{(a, f(a)) : a \in A\}$ that is total (every a appears) and deterministic (exactly one b per a). This connects Ch. 2 relations to Ch. 3 functions.

Example 3.3 (Counting functions). Number of functions $A \rightarrow B$ is $|B|^{|A|}$ (product rule: $|B|$ choices per a). Number of injections is $P(|B|, |A|)$ if $|A| \leq |B|$, else 0. Bijections = $|A|!$ when $|A| = |B|$.

Example 3.4 (Cantor's pairing). $\pi(m, n) = \frac{(m+n)(m+n+1)}{2} + m$ bijects $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ by enumerating anti-diagonals. Hence $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$.

A function is deterministic: same input always yields same output, unlike a relation that may branch. The vertical line test in calculus is the arrow picture in disguise: one output per input. Injections never collide, surjections never miss, bijections do both perfectly. Composition is function plumbing: output of one becomes input of the next. In programming, injective naming means two different variables never alias the same object. Counting injections $P(|B|, |A|)$ explains why pigeonhole forces collisions when $|A| > |B|$. Bijections are the gold standard for saying two sets have the same size, even when infinite. Cantor's diagonal is a prototype for impossibility proofs throughout computability. The Schröder–Bernstein theorem guarantees that mutual injections imply a bijection, though constructing it is non-trivial. Characteristic functions biject $\mathcal{P}(S)$ with

$\{0,1\}^S$, linking sets to binary strings. A function is deterministic: same input always yields same output, unlike a relation that may branch. The vertical line test in calculus is the arrow picture in disguise: one output per input. Injections never collide, surjections never miss, bijections do both perfectly. Composition is function plumbing: output of one becomes input of the next. In programming, injective naming means two different variables never alias the same object. Counting injections $P(|B|, |A|)$ explains why pigeonhole forces collisions when $|A| > |B|$. Bijections are the gold standard for saying two sets have the same size, even when infinite. Cantor's diagonal is a prototype for impossibility proofs throughout computability. The Schröder–Bernstein theorem guarantees that mutual injections imply a bijection, though constructing it is non-trivial. Characteristic functions biject $\mathcal{P}(S)$ with $\{0,1\}^S$, linking sets to binary strings. A function is deterministic: same input always yields same output, unlike a relation that may branch. The vertical line test in calculus is the arrow picture in disguise: one output per input. Injections never collide, surjections never miss, bijections do both perfectly. Composition is function plumbing: output of one becomes input of the next. In programming, injective naming means two different variables never alias the same object. Counting injections $P(|B|, |A|)$ explains why pigeonhole forces collisions when $|A| > |B|$. Bijections are the gold standard for saying two sets have the same size, even when infinite. Cantor's diagonal is a prototype for impossibility proofs throughout computability. The Schröder–Bernstein theorem guarantees that mutual injections imply a bijection, though constructing it is non-trivial. Characteristic functions biject $\mathcal{P}(S)$ with $\{0,1\}^S$, linking sets to binary strings. A function is deterministic: same input always yields same output, unlike a relation that may branch. The vertical line test in calculus is the arrow picture in disguise: one output per input. Injections never collide, surjections never miss, bijections do both perfectly. Composition is function plumbing: output of one becomes input of the next.

Theorem 3.3 (Schröder–Bernstein sketch). If injections $f : A \rightarrow B$ and $g : B \rightarrow A$ exist, chain-decompose $A \cup B$ into alternating f, g orbits to build a bijection.

Example 3.5 (Hash functions are not injective). A hash $h : \{0,1\}^* \rightarrow \{0,1\}^{256}$ cannot be injective (domain infinite, codomain finite) — collisions must exist; security relies on finding them being hard.

In programming, injective naming means two different variables never alias the same object. Counting injections $P(|B|, |A|)$ explains why pigeonhole forces collisions when $|A| > |B|$. Bijections are the gold standard for saying two sets have the same size, even when infinite. Cantor's diagonal is a prototype for impossibility proofs throughout computability. The Schröder–Bernstein theorem guarantees that mutual injections imply a bijection, though constructing it is non-trivial. Characteristic functions biject $\mathcal{P}(S)$ with $\{0,1\}^S$, linking sets to binary strings. A function is deterministic: same input always yields same output, unlike a relation that may branch. The vertical line test in calculus is the arrow picture in disguise: one output per input. Injections never collide, surjections never miss, bijections do both perfectly. Composition is function plumbing: output of one becomes input of the next. In programming, injective naming means two different variables never alias the same object. Counting injections $P(|B|, |A|)$ explains why pigeonhole forces collisions when $|A| > |B|$. Bijections are the gold standard for saying two sets have the same size, even when infinite.

3.4 Worked Example: Bijections in Action

Show $f : \mathbb{N} \rightarrow \mathbb{Z}$, $f(n) = \begin{cases} n/2 & n \text{ even} \\ -(n+1)/2 & n \text{ odd} \end{cases}$ is bijective. Injective: distinct n give distinct $f(n)$ because even/odd cases map to ≥ 0 vs. < 0 . Surjective: any $z \geq 0$ is $f(2z)$, any $z < 0$ is $f(-2z-1)$. Hence $|\mathbb{N}| = |\mathbb{Z}|$.

Contrast $g : \mathbb{R} \rightarrow \mathbb{R}$, $g(x) = x^2$ restricted to $[0, \infty) \rightarrow [0, \infty)$, $g(x) = x^2$ becomes bijective with $g^{-1}(y) = \sqrt{y}$. The domain restriction repairs injectivity (no $\pm x$ collision) and codomain restriction repairs surjectivity (no negative targets). This illustrates why both sets matter in the definition.

Cantor's theorem: for any set S , $|S| < |\mathcal{P}(S)|$. Diagonal again: assume $f : S \rightarrow \mathcal{P}(S)$ surjective, consider $D = \{x \in S : x \notin f(x)\}$; $D = f(d)$ for some d leads to $d \in D \iff d \notin D$, contradiction.

3.5 Chapter Notes

Dirichlet formalised functions; Cantor (1874–1891) founded cardinality theory. Bijections underlie hashing, cryptography permutations, and data-structure isomorphisms. Diagonalisation reappears in undecidability (SC2001).

3.6 Exercises

E3.1 **Classification.** For each $f : \mathbb{R} \rightarrow \mathbb{R}$ decide injective/surjective/bijective: (a) $x \mapsto x^3 - x$, (b) $x \mapsto e^x$, (c) $x \mapsto 2x + 3$. Prove each claim and, when bijective, give f^{-1} .

E3.2 **Composition.** Prove: if $g \circ f$ is injective then f is injective; if $g \circ f$ is surjective then g is surjective. Give counterexamples to the converses.

E3.3 **Finite bijections.** Let $|A| = n$. Show $f : A \rightarrow A$ injective $\Rightarrow$ surjective (pigeonhole) and count the number of bijections $A \rightarrow A$.

E3.4 **Countability.** Exhibit an explicit bijection $\mathbb{N} \rightarrow \mathbb{Z}$ and argue $\mathbb{Q}$ is countable by enumerating fractions p/q in lowest terms along diagonals (draw the grid).

E3.5 **Uncountability.** Use diagonalisation to prove $\{0, 1\}^\infty$ (infinite binary strings) is uncountable, and deduce $|\mathcal{P}(\mathbb{N})| > |\mathbb{N}|$ via characteristic functions.

Chapter 4

Proof: Direct, Contrapositive, Contradiction and Induction

“A proof is a story that convinces even a sceptical reader.” We systematise four story patterns that cover almost every theorem in discrete mathematics.

From zero: proving from scratch. We move from “why should I believe it?” to formal templates: assume premise $\rightarrow$ deduce conclusion (direct), flip to contrapositive, seek a clash (contradiction), and climb an infinite ladder (induction & strong induction).

Learning Outcomes

After this chapter you will be able to:

- (L1) structure direct proofs from definitions;
- (L2) prove $P \rightarrow Q$ via its contrapositive $\neg Q \rightarrow \neg P$;
- (L3) use contradiction to prove negations and impossibility;
- (L4) prove universal statements over $\mathbb{N}$ by weak and strong induction;
- (L5) spot faulty induction and fix base cases and inductive hypotheses.

4.1 Direct Proof

To prove $P \rightarrow Q$, assume P and derive Q using definitions, algebra, and prior theorems. Start by writing what P and Q mean.

Definition 4.1 (Even and odd). $n \in \mathbb{Z}$ is *even* if $n = 2k$ for some $k \in \mathbb{Z}$; *odd* if $n = 2k + 1$.

Example 4.1 (Direct). Claim: If n is odd then n^2 is odd. Proof: $n = 2k+1 \Rightarrow n^2 = 4k^2+4k+1 = 2(2k^2+2k)+1$, odd. □

Example 4.2 (Direct with divisibility). Claim: If $a \mid b$ and $b \mid c$ then $a \mid c$. Proof: $b = ak$, $c = b\ell = ak\ell$, so $a \mid c$.

Theorem 4.1 (Sum of two rationals is rational). If $x, y \in \mathbb{Q}$ then $x + y \in \mathbb{Q}$.

Proof. Write $x = p/q$, $y = r/s$ with integers, $q, s \neq 0$; then $x + y = (ps + rq)/qs \in \mathbb{Q}$. □

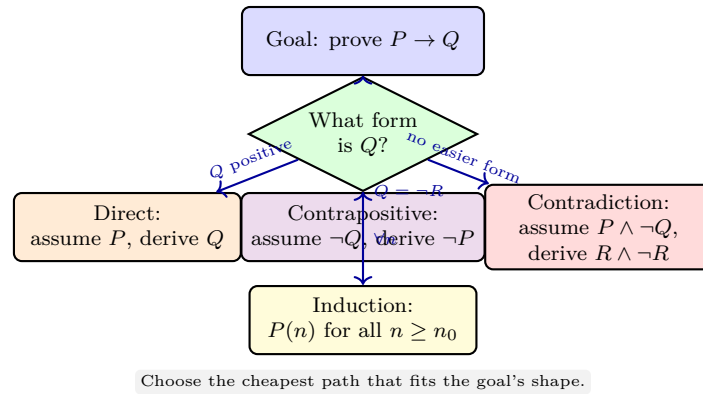


Figure 4.1: Choosing a proof strategy by inspecting the goal. Colour helps memorise: green decision, warm options.

4.2 Contrapositive

$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$. When Q is a negation or P messy, proving $\neg Q \rightarrow \neg P$ is cleaner — yet still a direct proof of the flipped statement.

Example 4.3. Claim: If n^2 is even then n is even. Direct is awkward (square root). Contrapositive: if n odd ($n = 2k + 1$) then n^2 odd (proved above). □

Lemma 4.1. If x^2 irrational implies x irrational? Contrapositive: rational $x = p/q \Rightarrow x^2 = p^2/q^2$ rational.

4.3 Contradiction

To prove P , assume $\neg P$ and derive $R \wedge \neg R$.

Theorem 4.2 (Irrationality of $\sqrt{2}$). $\sqrt{2} \notin \mathbb{Q}$.

Proof. Assume $\sqrt{2} = p/q$ in lowest terms, $\gcd(p, q) = 1$, $q > 0$. Then $2q^2 = p^2 \Rightarrow p$ even $= 2k \Rightarrow 2q^2 = 4k^2 \Rightarrow q^2 = 2k^2 \Rightarrow q$ even — both even contradicts $\gcd = 1$. □

Theorem 4.3 (Infinitely many primes). There are infinitely many primes.

Proof. Suppose $p_1, \dots, p_k$ were all primes. Let $N = p_1 \cdots p_k + 1$. N has a prime divisor p (proved by induction). p cannot be any p_i (remainder 1), contradiction. □

4.4 Induction and Strong Induction

Induction is dominoes: knock first, each knocks next.

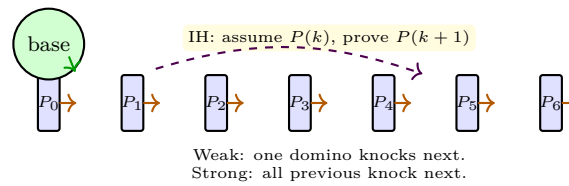


Figure 4.2: Induction as dominoes. Weak uses $P(k)$ alone; strong uses $P(n_0), \dots, P(k)$.

Definition 4.2 (Weak induction). To prove $\forall n \geq n_0 P(n)$: (Base) $P(n_0)$; (Step) $\forall k \geq n_0, P(k) \Rightarrow P(k+1)$.

Theorem 4.4 (Sum of first n integers). $\sum_{i=1}^n i = n(n+1)/2$ for $n \geq 1$.

Proof. Base $1 = 1 \cdot 2/2$. IH $\sum_{i=1}^k i = k(k+1)/2$. Then $\sum_{i=1}^{k+1} i = k(k+1)/2 + (k+1) = (k+1)(k+2)/2$. $\square$

Definition 4.3 (Strong induction). Assume $P(j)$ for all $n_0 \leq j \leq k$ to get $P(k+1)$.

Theorem 4.5 (Every $n \geq 2$ has a prime factor). Every integer $n \geq 2$ is divisible by a prime.

Proof. Base 2 prime. Fix $k \geq 2$, assume every $2 \leq j \leq k$ has prime divisor. If $k+1$ prime done; else $k+1 = ab$ with $2 \leq a, b \leq k$, prime divisor of a divides $k+1$. $\square$

Example 4.4 (Faulty induction). “All horses same colour”: false — the $P(k) \Rightarrow P(k+1)$ step fails at $k = 1$ (overlap needs ≥ 2 horses). Base $n = 1$ true, step invalid.

```

1 # Induction mirrors recursion: prove recursive function correct by induction on n
2 def fact(n):
3     if n==0: return 1          # base P(0)
4     return n*fact(n-1)        # step uses P(n-1) to get P(n)
5
6 # Strong induction: every n>=2 has prime divisor (constructive)
7 def prime_divisor(n):
8     for p in range(2, n+1):
9         if n%p==0 and all(p% d for d in range(2,p)):
10             return p
11 print([prime_divisor(n) for n in range(2,15)])

```

Remark 4.1 (Choosing weak vs. strong). Use strong when $P(k+1)$ needs not just $P(k)$ but an earlier $P(j)$ (divisors, recurrences with two steps, coin problem). Otherwise weak suffices.

Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible? Direct proof is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just

inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove. Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible? Direct proof is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove. Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible? Direct proof is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove. Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible?

Theorem 4.6 (Well-ordering principle equivalence). Weak induction, strong induction, and the well-ordering principle (every non-empty $S \subseteq \mathbb{N}$ has a least element) are equivalent.

Sketch. Well-ordering $\Rightarrow$ strong induction: if P fails, the set $\{n : P(n) \text{ false}\}$ has a least m , contradicting $P(j)$ for $j < m \Rightarrow P(m)$. The other directions are exercises. $\square$

Example 4.5 (Induction with inequality). Claim: $2^n > n^2$ for $n \geq 5$. Base $32 > 25$. IH $2^k > k^2$ for $k \geq 5$. Then $2^{k+1} = 2 \cdot 2^k > 2k^2 = k^2 + k^2 > k^2 + 2k + 1 = (k+1)^2$ since $k^2 > 2k + 1$ for $k \geq 3$.

Theorem 4.7 (Contradiction vs contrapositive). Contrapositive proves $P \rightarrow Q$ by proving $\neg Q \rightarrow \neg P$ (one flip). Contradiction assumes $P \wedge \neg Q$ and derives any absurdity — strictly more powerful but often overkill.

Example 4.6 (Irrational e or π sketch). Classical contradictions show e irrational via series remainder $< 1/n!$ squeeze; π is harder (Lambert).

Direct proof is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove. Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible? Direct proof

is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove. Every proof begins by unpacking definitions: what does it mean to be even, prime, or divisible? Direct proof is the workhorse: assume the left, derive the right, step by step. Contrapositive is useful when the conclusion is a negation, flipping the problem into a positive one. Contradiction is the tool of impossibility: assume the opposite and find an absurdity. Induction is the infinite domino chain, turning one base step and one inductive link into all cases. Strong induction is not stronger logically, just more convenient when the next case needs earlier ones. Well-ordering, weak induction, and strong induction are three faces of the same principle on $\mathbb{N}$. A faulty base case can sink an induction, as the horse colour paradox memorably shows. Loop invariants are just inductions over the number of iterations, connecting MH1812 to algorithm correctness. Writing the induction hypothesis explicitly prevents the common mistake of assuming what you need to prove.

4.5 Worked Example: A Complete Induction Proof Dissected

Prove $1 + 3 + 5 + \dots + (2n - 1) = n^2$. Let $P(n)$ be the equality. Base $P(1)$: $1 = 1^2$. IH $P(k)$: sum to k equals k^2 . Then sum to $k + 1$ is $k^2 + (2(k + 1) - 1) = k^2 + 2k + 1 = (k + 1)^2$, i.e. $P(k + 1)$. Every step is explicit: state P , check base, assume IH, derive next.

Strong induction example: prove every $n \geq 1$ has binary representation. Strong IH: all $j < n$ have binary. If n even, $n = 2k$ with $k < n$, take representation of k and append 0; if n odd, $n = 2k + 1$, take k and append 1. Base $1 = 1_2$. This needs both parities, hence all smaller j , not just $n - 1$.

4.6 Chapter Notes

Euclid pioneered indirect proof; induction was formalised by Pascal and Peano. Hoare logic and loop invariants (SC2001) are just induction over iterations. Strong induction = weak induction with strengthened hypothesis $Q(k) = \forall j \leq k P(j)$.

4.7 Exercises

E4.1 **Direct.** Prove: if m, n even then $m + n$ even; if $a \mid b$ and $a \mid c$ then $a \mid (b + c)$.

E4.2 **Contrapositive.** Prove: if n^2 divisible by 3 then n divisible by 3. Why is direct messy? Write the contrapositive explicitly.

E4.3 **Contradiction.** Prove $\sqrt{3}$ irrational and that there is no smallest positive rational (if $r > 0$ rational then

$r/2$ smaller positive rational — contradiction to minimality).

E4.4 **Induction.** Prove $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$ and $2^n > n$ for $n \geq 1$ by weak induction.

E4.5 **Strong induction.** Every $n \geq 12$ is $4a+5b$ for $a, b \geq 0$. Prove by strong induction with bases 12, 13, 14, 15. Where does weak fail?

Chapter 5

Counting: Product, Sum, Permutations, Combinations and Pigeonhole

“Counting is the oldest intellectual art.” Algorithms enumerate possibilities; combinatorics tells us how many there are — without listing them all.

From zero: choices as branches. Picture decisions as a tree: each level = a choice, each leaf = an outcome. We derive product/sum rules from tree width, then count orderings (permutations) and selections (combinations), and finally the pigeonhole principle that forces collisions.

Learning Outcomes

After this chapter you will be able to:

- (L1) apply product and sum rules to sequential and alternative choices;
- (L2) count permutations $P(n, r)$ and combinations $\binom{n}{r}$ and relate them;
- (L3) use complement and inclusion–exclusion to simplify counting;
- (L4) apply the pigeonhole principle and its generalised form;
- (L5) solve stars-and-bars and distribution problems.

5.1 Product and Sum Rules

Definition 5.1 (Product rule). If a procedure breaks into k sequential stages with n_i choices at stage i (independent of earlier choices), total outcomes = $n_1 n_2 \cdots n_k$.

Picture: a k -level tree where each node at level $i - 1$ sprouts n_i children — leaves = $\prod n_i$.

Definition 5.2 (Sum rule). If tasks $T_1, \dots, T_k$ are disjoint (no overlap), with n_i ways for T_i , total ways for “do some T_i ” = $\sum n_i$.

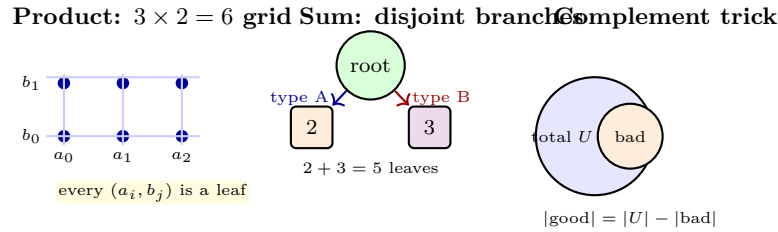


Figure 5.1: Product = grid, Sum = disjoint branches, Complement = subtract bad from universe.

Example 5.1. A password of 3 letters then 2 digits: $26^3 \cdot 10^2$ (product). A committee of 1 president OR 1 secretary: $n + m$ ways if disjoint pools (sum). Counting with “at least one” often uses complement: total minus “none”.

Lemma 5.1 (Subtraction / complement). $|A \setminus B| = |A| - |A \cap B|$; if $A \subseteq U$, $|\bar{A}| = |U| - |A|$.

5.2 Permutations and Combinations

Permutation $P(n, r)$: ordered arrangements of r distinct items from n :

$$P(n, r) = n(n-1) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

Tree: n choices for position 1, $n-1$ for 2, etc.

Combination $\binom{n}{r}$: unordered subsets of size r from n :

$$\binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!}.$$

Divide by $r!$ because each subset is permuted $r!$ ways internally.

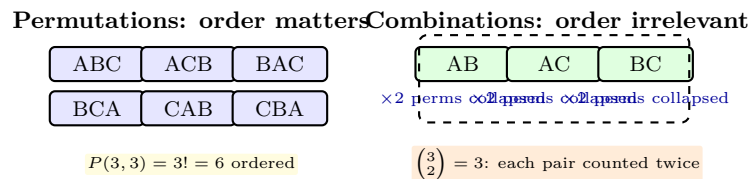


Figure 5.2: Permutations keep order; combinations collapse $r!$ orderings into one subset.

Theorem 5.1 (Binomial identities). $\binom{n}{r} = \binom{n}{n-r}$, $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$ (Pascal), $\sum_{r=0}^n \binom{n}{r} = 2^n$.

Proof. Combinatorial or algebraic: Pascal counts subsets containing a distinguished element vs. not. □

Pascal’s triangle visualises the recurrence; row n sums to 2^n (Ch. 2: subsets by size).

```

1 import math
2 def P(n,r): return math.factorial(n)//math.factorial(n-r)

```

```

3 def C(n,r): return math.factorial(n)//(math.factorial(r)*math.factorial(n-r))
4 print([C(5,r) for r in range(6)], sum(C(5,r) for r in range(6))) # 32 = 2^5
5 # Stars and bars: x1+...+xk=n, xi>=0 => C(n+k-1,k-1)
6 def starsBars(n,k): return C(n+k-1,k-1)
7 print(starsBars(10,3))

```

5.3 Pigeonhole Principle

If n pigeons go into m holes and $n > m$, some hole has ≥ 2 pigeons. Generalised: some hole has $\geq \lceil n/m \rceil$.

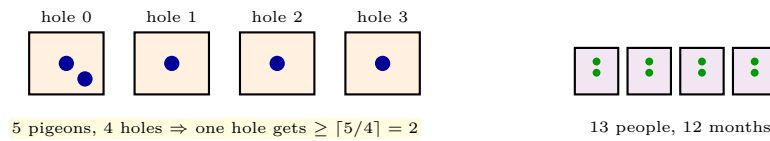


Figure 5.3: Pigeonhole: more items than boxes forces a collision. Generalised quantifies the overload.

Example 5.2. Among 13 people, two share a birth month ($13 > 12$). In any $n + 1$ integers, two differ by a multiple of n (remainders mod n are n holes).

Theorem 5.2 (Stars and bars). Number of k -tuples $(x_1, \dots, x_k)$ with $x_i \geq 0$ integers and $\sum x_i = n$ is $\binom{n+k-1}{k-1}$. Picture n stars and $k - 1$ bars.

Inclusion–exclusion extends complement: $|A \cup B| = |A| + |B| - |A \cap B|$, $|A \cup B \cup C| = \sum |A_i| - \sum |A_i \cap A_j| + |A \cap B \cap C|$.

Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars. Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$. Complement counting often simplifies at least one or at least two problems dramatically. Binomial theorem expands powers by counting how many factors contribute each variable. Lattice path counting shows combinatorics is geometry: every path is a binary string of rights and ups. Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars. Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$. Complement counting often simplifies at least one or at least two problems dramatically. Binomial theorem expands powers by counting how many factors contribute each variable. Lattice path counting shows combinatorics is geometry: every path is a binary string of rights and ups. Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations

count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars. Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$. Complement counting often simplifies at least one or at least two problems dramatically. Binomial theorem expands powers by counting how many factors contribute each variable. Lattice path counting shows combinatorics is geometry: every path is a binary string of rights and ups. Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars.

Remark 5.1 (Overcounting pitfall). Counting ordered then dividing by symmetry is powerful but error-prone: ensure each object is overcounted equally. For “choose a pair then an extra” counts, check if the extra could have been in the pair.

Example 5.3 (Committee with overlap). Choose 3 from 10 then a leader among 3: $\binom{10}{3} \cdot 3$. Alternative: choose leader first (10 ways) then $\binom{9}{2}$ others: $10\binom{9}{2}$ — same count. Two viewpoints, one answer.

Theorem 5.3 (Binomial theorem). $(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$. Combinatorial: choose k factors contributing y .

Example 5.4 (Lattice paths). Monotone paths from $(0, 0)$ to (m, n) moving right/up: $\binom{m+n}{m}$ (choose which steps are right).

Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$. Complement counting often simplifies at least one or at least two problems dramatically. Binomial theorem expands powers by counting how many factors contribute each variable. Lattice path counting shows combinatorics is geometry: every path is a binary string of rights and ups. Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars. Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$. Complement counting often simplifies at least one or at least two problems dramatically. Binomial theorem expands powers by counting how many factors contribute each variable. Lattice path counting shows combinatorics is geometry: every path is a binary string of rights and ups. Product counts sequential choices, sum counts alternative choices, together they build most counts. Tree diagrams make product and sum visible: branching width multiplies, separate trees add. Permutations count orderings, combinations count selections, the $r!$ factor is the bridge between them. Pascal's recurrence builds binomial coefficients from smaller ones, mirroring combinatorial reasoning. Stars and bars turns integer compositions into placements of separators among stars. Inclusion–exclusion generalises the Venn lens correction to any number of overlapping sets. Pigeonhole is the Dirichlet principle: more items than boxes guarantees a collision, quantified by $\lceil n/m \rceil$.

5.4 Worked Example: Two Ways to Count

Count the number of binary strings of length n with exactly k ones: $\binom{n}{k}$ (choose positions for ones). Now count the same set by complement: strings with $\geq k$ ones = total 2^n minus strings with $< k$ ones. For $n = 5, k = 3$, $\binom{5}{3} = 10$, and $\sum_{i=0}^2 \binom{5}{i} = 1 + 5 + 10 = 16$, so ≥ 3 has $32 - 16 = 16$; indeed $\binom{5}{3} + \binom{5}{4} + \binom{5}{5} = 10 + 5 + 1 = 16$.

Stars and bars worked: distribute 10 identical cookies to 3 children. Put 10 stars in a row, insert 2 bars to split into 3 blocks: e.g. $***|****|***$ means 3, 4, 3. Number of placements of 2 bars among 12 positions = $\binom{12}{2} = 66$. If each child must get ≥ 1 , pre-give 1 each, then distribute 7 remaining: $\binom{9}{2} = 36$.

5.5 Chapter Notes

Combinatorics predates computers but underpins complexity counting and probability analysis of algorithms. Binomial coefficients appear in complexity bounds and generating functions (Ch. 6). The pigeonhole principle appears everywhere from hashing collisions to lower bounds.

5.6 Exercises

E5.1 **Product/sum.** How many 4-character strings with 2 letters then 2 digits have no repeated character? How many have at least one digit repeated? Use complement.

E5.2 **Permutations vs combinations.** A team of 3 from 10 plus a leader among the 3: count two ways and explain the factor 3.

E5.3 **Binomial.** Prove $\binom{n}{r} = \binom{n}{n-r}$ combinatorially and verify Pascal $\binom{5}{2} = \binom{4}{1} + \binom{4}{2}$ via counting.

E5.4 **Pigeonhole.** Show among any 5 points in a unit square, two are at distance $\leq \sqrt{2}/2$ (partition into 4 quarter-squares).

E5.5 **Stars and bars.** Count solutions to $x_1 + x_2 + x_3 = 10$ with $x_i \geq 0$; then with $x_i \geq 1$ (shift $y_i = x_i - 1$). Relate to $\binom{n}{r}$.

Chapter 6

Recurrences: Induction Proofs and Closed Forms

“To understand recursion, you must first understand recursion.” Recurrence relations describe sequences where each term depends on earlier ones — the mathematical shadow of every recursive program.

From zero: sequences that define themselves. We start with “next = function of previous” (Fibonacci, factorial, Hanoi), see why naive recursion explodes, then solve recurrences to closed forms and prove them by induction.

Learning Outcomes

After this chapter you will be able to:

- (L1) write recurrences from recursive algorithms and combinatorial descriptions;
- (L2) solve linear recurrences by iteration/expansion and characteristic equations;
- (L3) guess and verify closed forms by induction (including strong);
- (L4) analyse divide-and-conquer recurrences informally;
- (L5) connect recurrences to induction proofs and generating functions (preview).

6.1 What Is a Recurrence?

A *recurrence* (or recursion) defines a_n in terms of earlier a_k ($k < n$) plus base cases. Example: factorial $a_0 = 1$, $a_n = n \cdot a_{n-1}$; Fibonacci $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$; Tower of Hanoi $H_0 = 0, H_n = 2H_{n-1} + 1$.

Recursive algorithms induce recurrences on running time: e.g. binary search $T(n) = T(n/2) + O(1)$.

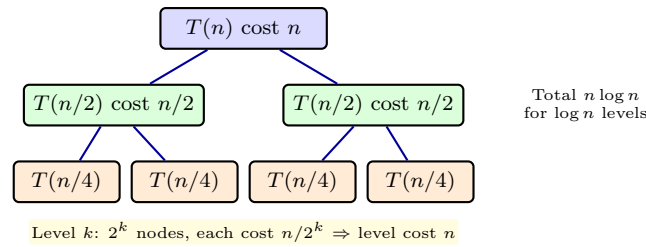


Figure 6.1: Recursion tree for $T(n) = 2T(n/2) + n$. Summing level costs gives n per level $\times \log n$ levels.

Solving means finding a *closed form* with no recursion, e.g. $H_n = 2^n - 1$.

6.2 Solving by Iteration (Unrolling)

Unroll until pattern emerges, sum the series, prove by induction.

Example 6.1 (Hanoi). $H_n = 2H_{n-1} + 1$, $H_0 = 0$. Then $H_n = 2(2H_{n-2} + 1) + 1 = 2^2H_{n-2} + 2 + 1 = \dots = 2^n H_0 + \sum_{i=0}^{n-1} 2^i = 2^n - 1$.

Theorem 6.1 (Arithmetic recurrence). $a_n = a_{n-1} + c$, a_0 given $\Rightarrow a_n = a_0 + cn$.

Theorem 6.2 (Geometric + constant). $a_n = r \cdot a_{n-1} + c$, $r \neq 1 \Rightarrow a_n = r^n a_0 + c \frac{r^n - 1}{r - 1}$ (geometric sum).

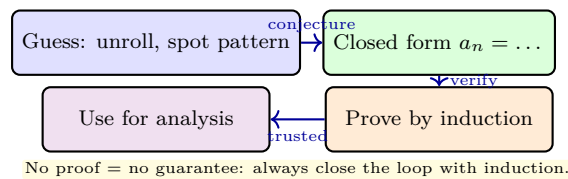


Figure 6.2: Workflow: iterate to conjecture, induct to certify.

Induction proofs for recurrences are routine: base checks closed form at n_0 ; step substitutes IH into recurrence to get $n + 1$ case.

6.3 Linear Homogeneous Recurrences

For $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ (order 2, constant coefficients), try $a_n = r^n$: characteristic equation $r^2 - c_1 r - c_2 = 0$ with roots r_1, r_2 .

Theorem 6.3 (Second-order linear). If $r_1 \neq r_2$ then $a_n = \alpha r_1^n + \beta r_2^n$ for constants α, β from bases. If $r_1 = r_2 = r$ then $a_n = (\alpha + \beta n) r^n$.

Example 6.2 (Fibonacci closed form (Binet)). $F_n = F_{n-1} + F_{n-2}$, $r^2 = r + 1 \Rightarrow r = (1 \pm \sqrt{5})/2$. So $F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$. Exponential growth $\approx \phi^n / \sqrt{5}$.

Example 6.3 (Non-homogeneous). $a_n = 2a_{n-1} + 1$ has homogeneous part $A2^n$ plus particular constant $a = -1$: $a_n = A2^n - 1$, base gives $A = 1$ for $a_0 = 0$.

```

1 # Verify closed form vs recurrence by induction check (testing)
2 def H_rec(n): return 0 if n==0 else 2*H_rec(n-1)+1
3 def H_closed(n): return 2**n - 1
4 print([H_rec(n)==H_closed(n) for n in range(10)])
5
6 # Fibonacci via recurrence vs Binet
7 import math
8 phi=(1+5**0.5)/2; psi=(1-5**0.5)/2
9 def fib_closed(n): return round((phi**n - psi**n)/5**0.5)
10 fib=lambda n: 0 if n==0 else 1 if n==1 else fib(n-1)+fib(n-2)
11 print([fib(n)==fib_closed(n) for n in range(12)])

```

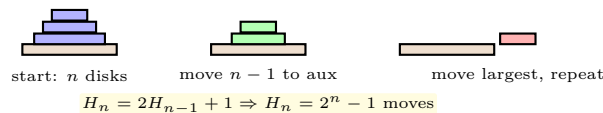


Figure 6.3: Tower of Hanoi intuition: to move n disks, move $n - 1$ aside, move largest, move $n - 1$ back.

Remark 6.1 (Master preview). Divide-and-conquer $T(n) = aT(n/b) + f(n)$ has similar tree analysis; full Master theorem appears in SC2001. The iteration idea already predicts $n^{\log_b a}$ vs. $f(n)$ dominance.

Theorem 6.4 (Telescoping sum view). Unrolling $a_n = ra_{n-1} + c$ yields $a_n = r^n a_0 + c \sum_{i=0}^{n-1} r^i$. This is the discrete analogue of solving a linear ODE.

Example 6.4 (Compound interest). $A_n = (1 + r)A_{n-1}$ with A_0 principal gives $A_n = A_0(1 + r)^n$ — pure geometric recurrence.

Theorem 6.5 (First-order linear general). $a_n = p_n a_{n-1} + q_n$ with $p_n \neq 0$ solves to $a_n = (\prod_{i=1}^n p_i) a_0 + \sum_{k=1}^n q_k \prod_{i=k+1}^n p_i$ by forward substitution.

6.4 Induction Proofs for Closed Forms

Every closed form must be certified by induction because pattern-spotting is not proof. Template: let $P(n)$ be “ $a_n = \text{closed}(n)$ ”. Base $P(n_0)$ by direct check; inductive step $P(k) \Rightarrow P(k+1)$ by substituting $P(k)$ into the recurrence and algebra.

Example 6.5 (Detailed induction). Claim: $a_n = 2^n - 1$ solves $a_0 = 0$, $a_n = 2a_{n-1} + 1$. Base $a_0 = 1 - 1 = 0$. IH $a_k = 2^k - 1$. Then $a_{k+1} = 2a_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$.

Remark 6.2 (Strong induction for order 2). For $F_n = F_{n-1} + F_{n-2}$, both $P(k)$ and $P(k-1)$ are needed, so use strong induction or weak with strengthened hypothesis $Q(k) = P(k) \wedge P(k-1)$.

```

1 # Strong induction check for Fibonacci identity sum F_i = F_{n+2}-1
2 def fib(n):
3     a,b=0,1
4     for _ in range(n): a,b=b,a+b
5     return a
6 for n in range(10):
7     assert sum(fib(i) for i in range(n+1)) == fib(n+2)-1
8 print("Fib identity holds for n<10")

```

Example 6.6 (Second-order with repeated root detail). $a_n = 4a_{n-1} - 4a_{n-2}$, $a_0 = 1, a_1 = 4$: $r^2 - 4r + 4 = (r - 2)^2 \Rightarrow r = 2$ double. So $a_n = (\alpha + \beta n)2^n$. Bases: $1 = \alpha$, $4 = (\alpha + \beta)2 \Rightarrow \beta = 1$, hence $a_n = (1 + n)2^n$.

Theorem 6.6 (Particular solution for polynomial forcing). If $a_n = c_1 a_{n-1} + c_2 a_{n-2} + p(n)$ with p polynomial degree d , try particular $q(n)$ polynomial degree d (times n^k if root collision).

Remark 6.3 (Why induction after guessing is mandatory). Unrolling suggests $a_n = 2^n - 1$ but does not prove it — pattern could break at $n = 100$ (e.g. $n^2 + n + 41$ prime trap). Induction closes the gap.

```

1  # Solving linear recurrence via characteristic roots (numeric)
2  import cmath
3  def solve_order2(c1,c2,a0,a1,n):
4      # a_n = c1*a_{n-1}+c2*a_{n-2}
5      disc=c1*c1+4*c2
6      r1=(c1+cmath.sqrt(disc))/2
7      r2=(c1-cmath.sqrt(disc))/2
8      if abs(r1-r2)<1e-9:
9          r=r1
10         # a_n = (alpha+beta*n)*r^n
11         alpha=a0
12         beta=(a1/r - alpha) if r!=0 else 0
13         return (alpha+beta*n)*(r**n)
14     else:
15         # solve alpha+beta=a0, alpha*r1+beta*r2=a1
16         beta=(a1-a0*r1)/(r2-r1)
17         alpha=a0-beta
18         return alpha*(r1**n)+beta*(r2**n)
19 print([round(solve_order2(1,1,0,1,n).real) for n in range(10)])

```

Recurrences are the discrete analogue of differential equations, describing next from now. Unrolling reveals pattern, but only induction certifies the closed form for all n . Hanoi's $2^n - 1$ moves show how exponential growth emerges from a simple $2T(n-1) + 1$ rule. Arithmetic recurrences add a constant each step, geometric ones multiply, mixed ones do both. Characteristic equations turn linear recurrences into polynomial roots, an algebraic shortcut. Repeated roots need an extra n factor, reflecting the degeneracy of the trial solution. Particular solutions handle forcing terms like constants or polynomials, completing the general solution. Generating functions encode the whole sequence in a power series, a uniform method for all linear recurrences. Divide-and-conquer recurrences foreshadow the Master theorem, central to algorithm analysis. Fibonacci's closed form via ϕ^n shows linear recurrences can hide irrational numbers that cancel to integers. Recurrences are the discrete analogue of differential equations, describing next from now. Unrolling reveals pattern, but only induction certifies the closed form for all n . Hanoi's $2^n - 1$ moves show how exponential growth emerges from a simple $2T(n-1) + 1$ rule.

Example 6.7 (Merge sort recurrence). $T(n) = 2T(n/2) + O(n)$ unrolls to $O(n \log n)$ — the recursion tree of Ch. 6 predicts the SC2001 Master theorem case 2.

Theorem 6.7 (Generating function preview). Recurrence $a_n = 2a_{n-1} + 1$ has OGF $A(x) = \sum a_n x^n = \frac{x}{(1-2x)(1-x)}$; partial fractions recover $2^n - 1$.

Arithmetic recurrences add a constant each step, geometric ones multiply, mixed ones do both. Characteristic equations turn linear recurrences into polynomial roots, an algebraic shortcut. Repeated roots need an extra n

factor, reflecting the degeneracy of the trial solution. Particular solutions handle forcing terms like constants or polynomials, completing the general solution. Generating functions encode the whole sequence in a power series, a uniform method for all linear recurrences. Divide-and-conquer recurrences foreshadow the Master theorem, central to algorithm analysis. Fibonacci's closed form via ϕ^n shows linear recurrences can hide irrational numbers that cancel to integers. Recurrences are the discrete analogue of differential equations, describing next from now. Unrolling reveals pattern, but only induction certifies the closed form for all n . Hanoi's $2^n - 1$ moves show how exponential growth emerges from a simple $2T(n-1) + 1$ rule. Arithmetic recurrences add a constant each step, geometric ones multiply, mixed ones do both.

6.5 Worked Example: Solving a Non-Homogeneous Recurrence End-to-End

Solve $a_0 = 2$, $a_n = 3a_{n-1} + 4$ ($n \geq 1$). Iteration: $a_n = 3^n \cdot 2 + 4 \sum_{i=0}^{n-1} 3^i = 2 \cdot 3^n + 4 \frac{3^n - 1}{2} = 2 \cdot 3^n + 2(3^n - 1) = 4 \cdot 3^n - 2$. Prove by induction: base $n = 0$ gives $4 - 2 = 2$ ok. IH $a_k = 4 \cdot 3^k - 2$, then $a_{k+1} = 3(4 \cdot 3^k - 2) + 4 = 4 \cdot 3^{k+1} - 6 + 4 = 4 \cdot 3^{k+1} - 2$. Works.

Compare to homogeneous $b_n = 3b_{n-1}$ which would be $b_n = 2 \cdot 3^n$. The constant 4 adds a particular solution -2 (solve $A = 3A + 4 \Rightarrow A = -2$). General = homogeneous + particular.

6.6 Chapter Notes

Recurrences model recursive code, population growth, and financial compounding. Characteristic equations are linear algebra in disguise (eigenvalues). Generating functions (not covered) give a uniform solver.

6.7 Exercises

- E6.1 **Unrolling.** Solve $a_n = 3a_{n-1} + 2$, $a_0 = 1$ by iteration; prove your closed form by induction. Do the same for $a_n = a_{n-1} + n$, $a_0 = 0$.
- E6.2 **Hanoi proof.** Prove by induction that $H_n = 2^n - 1$ is minimal. Show any solution needs at least $2^n - 1$ moves (the largest disk must move once).
- E6.3 **Characteristic equation.** Solve $a_n = 5a_{n-1} - 6a_{n-2}$ with $a_0 = 0$, $a_1 = 1$. Solve $a_n = 4a_{n-1} - 4a_{n-2}$ with $a_0 = 1$, $a_1 = 4$ (repeated root).
- E6.4 **Fibonacci identities.** Prove $F_0 + \cdots + F_n = F_{n+2} - 1$ and $F_n \geq 2^{n/2-1}$ by induction (strong where needed).
- E6.5 **Divide-and-conquer.** Let $T(1) = 1$, $T(n) = 2T(n/2) + n$ for n power of 2. Unroll to guess $T(n) = n \log_2 n + n$, prove by induction.

Chapter 7

Graphs: Isomorphisms, Euler, Hamilton and Trees

“The bridges of Königsberg are the birthplace of graph theory.” Graphs model networks, dependencies, maps, and state spaces — vertices and edges are enough.

From zero: dots and lines. A graph is just circles (vertices) joined by lines (edges). We formalise degrees, paths, cycles, then ask: when are two drawings the same graph (isomorphism)? Can we walk each edge once (Euler) or visit each vertex once (Hamilton)? What makes a tree special?

Learning Outcomes

After this chapter you will be able to:

- (L1) define graph, degree, path, cycle, connectivity, and graph types;
- (L2) test graph isomorphism via invariants and exhibit bijections;
- (L3) characterise Eulerian trails/circuits via parity and apply Hierholzer/Königsberg;
- (L4) distinguish Euler vs. Hamilton and reason about NP-hardness intuition;
- (L5) characterise trees equivalently and count edges $m = n - 1$.

7.1 Graphs and Degrees

A graph $G = (V, E)$ has finite vertex set V and edge set $E \subseteq \{\{u, v\} : u \neq v\}$ for simple undirected graphs. Loops and multi-edges give multigraphs; ordered pairs give digraphs. Degree $\deg(v) = |\{e : v \in e\}|$.

Theorem 7.1 (Handshaking lemma). $\sum_{v \in V} \deg(v) = 2|E|$. Every edge contributes 2 to the degree sum. Hence number of odd-degree vertices is even.

Example 7.1 (Degrees). In K_n (complete graph) each $\deg = n - 1$ and $|E| = \binom{n}{2}$. In C_n (cycle) each $\deg = 2$, $|E| = n$. In Q_3 (3-cube) each $\deg = 3$, $|E| = 12$.

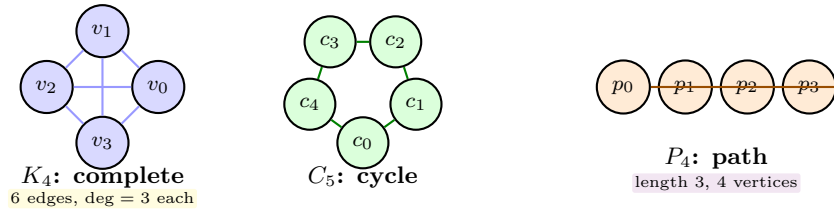


Figure 7.1: Three archetypes: complete, cycle, path. Colour distinguishes families.

A *path* $v_0v_1 \dots v_k$ has distinct vertices (except possibly ends). A *cycle* is a closed path $v_0 = v_k$. G is *connected* if a path exists between every pair. Component = maximal connected subgraph.

7.2 Isomorphism

$G \cong H$ if a bijection $f : V(G) \rightarrow V(H)$ preserves adjacency: $\{u, v\} \in E(G) \iff \{f(u), f(v)\} \in E(H)$. Isomorphic graphs are the same up to relabelling.

Invariants (if $G \cong H$, these match): $|V|, |E|$, degree sequence sorted, number of cycles of length k , connectivity, planarity. Mismatch proves non-isomorphic; matching does not prove isomorphic.

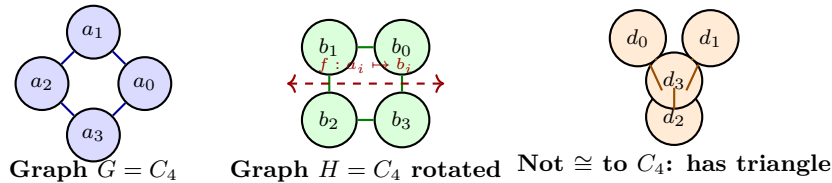


Figure 7.2: Isomorphism is a relabelling that preserves edges. Invariants like “contains a triangle” distinguish non-isomorphic graphs.

Example 7.2 (Non-isomorphic despite same degrees). Two 6-vertex 3-regular graphs exist: $K_{3,3}$ vs. prism. Both have $|V| = 6, |E| = 9$, all $\deg = 3$, but one is bipartite (no odd cycle), the other has a triangle — not isomorphic.

7.3 Euler Trails and Circuits

An *Euler circuit* traverses each edge exactly once and returns to start (closed trail). An *Euler trail* does so without returning.

Theorem 7.2 (Euler, 1736). A connected graph has an Euler circuit iff every vertex has even degree. It has an Euler trail (non-closed) iff exactly 0 or 2 vertices have odd degree ($0 \rightarrow$ circuit, $2 \rightarrow$ trail between the odd vertices).

Sketch. Even degrees allow pairing entry/exit at each vertex to splice cycles (Hierholzer). Odd count $\neq 0, 2$ makes pairing impossible. $\square$

Königsberg's 4 landmasses with 7 bridges had 4 odd-degree vertices — no Euler trail, hence the famous impossibility.

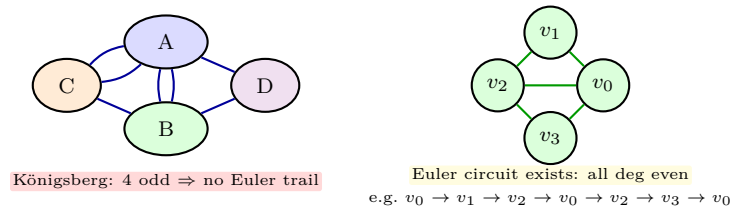


Figure 7.3: Left: Königsberg model (multigraph). Right: a graph with all even degrees and an explicit Euler circuit.

Hierholzer's algorithm finds an Euler circuit in $O(|E|)$: walk until stuck (must be at start by even degrees), then splice in cycles from unused edges.

7.4 Hamilton Cycles and Trees

A *Hamilton cycle* visits each vertex exactly once and returns; a *Hamilton path* visits each once. Unlike Euler (easy parity check), Hamilton is NP-complete — no simple characterisation is known.

Theorem 7.3 (Dirac). If $n \geq 3$ and every $\deg(v) \geq n/2$ then G is Hamiltonian (sufficient, not necessary).

A *tree* is a connected acyclic graph.

Theorem 7.4 (Tree equivalences). For G with n vertices, TFAE: (i) G is a tree; (ii) G connected and $|E| = n-1$; (iii) G acyclic and $|E| = n-1$; (iv) unique path between every two vertices.

Proof. (i) $\Rightarrow$ (ii): induction by removing a leaf (a tree has ≥ 2 leaves). Counts telescope to $n-1$. Other implications by similar leaf/edge counting. $\square$

```

1 # Handshaking and Euler check
2 def has_euler_circuit(degrees): return all(d%2==0 for d in degrees)
3 def has_euler_trail(degrees): return sum(d%2 for d in degrees) in (0,2)
4 print(has_euler_circuit([2,2,2,2]), has_euler_trail([1,1,2,2]))
5
6 # Tree check: connected (BFS) + edges == n-1 and acyclic
7 def is_tree(n, edges):
8     if len(edges)!=n-1: return False
9     from collections import deque, defaultdict
10    g=defaultdict(list)
11    for u,v in edges:
12        g[u].append(v); g[v].append(u)
13    seen={0}; q=deque([0])
14    while q:
15        u=q.popleft()
16        for w in g[u]:
17            if w not in seen: seen.add(w); q.append(w)
18    return len(seen)==n
19 print(is_tree(4, [(0,1),(1,2),(2,3)]))

```

Example 7.3 (Degree sequence graphic?). $(3, 3, 2, 2, 2)$ sums to $12 = 2 \cdot 6$, so handshaking allows it (needs 6 edges). But $(3, 3, 3, 1)$ sums to 10, also even, yet no simple graph realises it (Erdős–Gallai fails). Sum even is necessary, not sufficient.

Theorem 7.5 (Euler’s formula preview). For connected planar graphs, $V - E + F = 2$. Trees have $F = 1$ (outer face), so $E = V - 1$, matching the tree theorem.

Remark 7.1 (Bipartite test via BFS). G bipartite iff no odd cycle (equivalently 2-colourable). BFS 2-colouring in $O(V + E)$ tests this — used for $K_{3,3}$ vs. prism distinction.

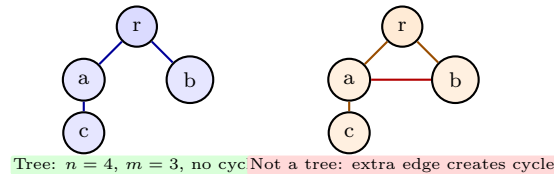


Figure 7.4: Tree vs. non-tree: one extra edge beyond $n - 1$ forces a cycle.

Handshaking lemma is a global invariant from local degrees, a double-counting gem. Complete graphs K_n maximise edges, paths P_n minimise connectivity, cycles C_n sit in between. Isomorphism is graph relabelling: invariants can disprove it, but only an explicit bijection proves it. Degree sequence, cycle counts, and bipartiteness are quick isomorphism filters before brute force. Euler’s parity condition is a rare easy characterisation: all even degrees versus NP-hard Hamilton.

Theorem 7.6 (Handshake for digraphs). $\sum_v \text{indeg}(v) = \sum_v \text{outdeg}(v) = |E|$ for digraphs; the undirected version follows by splitting each undirected edge into two arcs.

Example 7.4 (Bipartite vs odd cycle). C_3 (triangle) is not bipartite; any graph with an odd cycle fails 2-colouring. $K_{3,3}$ is bipartite yet non-planar (utility puzzle).

Königsberg’s four odd vertices made the walk impossible, birthing graph theory in 1736. Hierholzer’s algorithm efficiently stitches cycles into an Euler circuit in linear time. Hamilton cycles remain elusive: Dirac’s $n/2$ condition is sufficient but far from necessary. Trees are minimally connected and maximally acyclic, with exactly $n - 1$ edges and leaves to peel inductively. BFS trees and DFS trees link MH1812 graphs to traversal algorithms in later modules. Handshaking lemma is a global invariant from local degrees, a double-counting gem. Complete graphs K_n maximise edges, paths P_n minimise connectivity, cycles C_n sit in between. Isomorphism is graph relabelling: invariants can disprove it, but only an explicit bijection proves it. Degree sequence, cycle counts, and bipartiteness are quick isomorphism filters before brute force. Euler’s parity condition is a rare easy characterisation: all even degrees versus NP-hard Hamilton. Königsberg’s four odd vertices made the walk impossible, birthing graph theory in 1736.

7.5 Worked Example: Euler vs. Hamilton on the Same Graph

Consider C_5 (5-cycle). All degrees 2 even, so Euler circuit exists: walk around. Hamilton cycle also exists (the cycle itself). Now add a chord v_0v_2 : degrees become $3, 3, 2, 2, 2$ — two odds remain, so an Euler trail exists but not a circuit. Hamiltonian? $v_0v_1v_2v_3v_4v_0$ still works (use outer cycle). Finally $K_{2,3}$: degrees $3, 3, 2, 2, 2$ again, bipartite, Euler trail exists, Hamiltonian? $K_{2,3}$ has 5 vertices, any Hamilton cycle needs even length to alternate parts, so none exists (part sizes differ by 1). Invariants separate the concepts.

7.6 Chapter Notes

Euler founded graph theory (1736); Hamilton's game (1857) became the TSP. Trees link to induction (leaf removal), to recursion (DFS/BFS trees), and to minimum spanning trees (SC2001). Graph isomorphism is in NP but not known to be NP-complete (Babai's quasi-polynomial).

7.7 Exercises

- E7.1 **Handshaking.** Prove the number of odd-degree vertices is even. Give a graph with degree sequence $(3, 3, 2, 2, 2)$ or explain why impossible.
- E7.2 **Isomorphism.** Determine if the two 4-vertex graphs P_4 and C_4 are isomorphic via invariants. Find an explicit isomorphism between two drawn C_4 's or prove none exists for C_4 vs. diamond.
- E7.3 **Euler.** Which of K_5 , $K_{3,3}$, C_n , Q_3 have Euler circuits/trails? Apply the parity theorem and, when they exist, give the trail.
- E7.4 **Hamilton vs Euler.** Give a graph with an Euler circuit but no Hamilton cycle, and one with a Hamilton cycle but no Euler circuit. Explain.
- E7.5 **Trees.** Prove a tree with $n \geq 2$ has at least two leaves. Use this to prove by induction that a tree has $n - 1$ edges and a unique path between any two vertices.

Chapter 8

Number Theory: Divisibility, GCD, Modular Arithmetic and RSA Intuition

“God made the integers, all else is the work of man.” — Leopold Kronecker. Integers under division and remainders hide the mathematics behind cryptography.

From zero: sharing and remainders. Start with “ a divides b ” as exact sharing, build the division algorithm (quotient + remainder), compute greatest common divisors (Euclid), work on clocks (modular arithmetic), and glimpse how RSA turns these ideas into secrecy.

Learning Outcomes

After this chapter you will be able to:

- (L1) define divisibility, prime, composite, and apply the division algorithm;
- (L2) compute gcd and lcm via Euclid and prime factorisation;
- (L3) solve linear congruences and compute modular inverses when they exist;
- (L4) state and apply Fermat’s little theorem and Chinese remainder intuition;
- (L5) explain RSA key generation, encryption and decryption at a high level.

8.1 Divisibility and the Division Algorithm

For $a, b \in \mathbb{Z}$, $a \mid b$ (“ a divides b ”) if $\exists k \in \mathbb{Z} b = ak$. Then a is a divisor, b a multiple. Every $n \neq 0$ divides 0; $a \mid b \wedge b \mid a \Rightarrow |a| = |b|$.

Theorem 8.1 (Division algorithm). For $a \in \mathbb{Z}$, $d \in \mathbb{Z}^+$ there exist unique q, r with $a = dq + r$ and $0 \leq r < d$. q is quotient, $r = a \bmod d$ remainder.

Proof. Existence by well-ordering on $\{a - dk \geq 0\}$; uniqueness by gap $< d$. $\square$

A *prime* $p \geq 2$ has divisors $1, p$ only; *composite* otherwise. 1 is neither. Fundamental theorem (proved via Euclid's lemma): every $n \geq 2$ factors uniquely into primes.

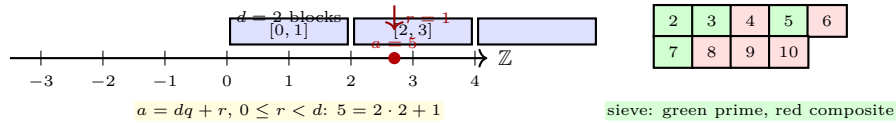


Figure 8.1: Division algorithm partitions $\mathbb{Z}$ into blocks of length d ; remainder r is offset inside the block.

Lemma 8.1 (Divisibility lemmas). If $a \mid b$ and $a \mid c$ then $a \mid (bx + cy)$ for any $x, y \in \mathbb{Z}$ (linear combination). If $a \mid b$ and $b \neq 0$ then $|a| \leq |b|$.

8.2 GCD and Euclid

$\gcd(a, b)$ is the greatest d with $d \mid a$ and $d \mid b$; $\text{lcm}(a, b)$ the least positive common multiple. $\gcd(0, b) = |b|$; $\gcd(a, b) = \gcd(b, a \bmod b)$.

Euclid's algorithm iterates $(a, b) \mapsto (b, a \bmod b)$ until $b = 0$; variant strictly decreases, gcd invariant, $O(\log \min(a, b))$ steps because remainder $< a/2$ when $b \leq a$.

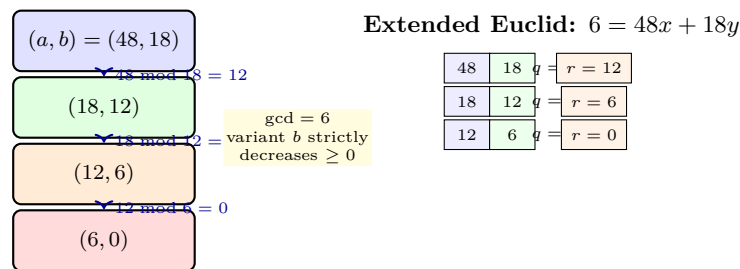


Figure 8.2: Euclid's algorithm: repeated remainder steps; extended version finds x, y with $ax + by = \gcd(a, b)$.

Theorem 8.2 (Bézout). $\gcd(a, b)$ is the smallest positive linear combination $ax + by$. Hence a, b coprime ($\gcd = 1$) iff $\exists x, y$ $ax + by = 1$.

Extended Euclid finds such x, y by back-substitution; crucial for modular inverses.

```

1 def egcd(a,b):
2     if b==0: return (a,1,0)
3     g,x1,y1=egcd(b, a % b)
4     return (g, y1, x1 - (a//b)*y1)
5
6 print(egcd(48,18)) # (6, -1, 3) => -1*48 + 3*18 = 6
7 def inv_mod(a,m):
8     g,x,y=egcd(a,m)
9     assert g==1, "no inverse"
10    return x % m
11 print(inv_mod(3,11)) # 4 because 3*4=12=1 mod 11

```

8.3 Modular Arithmetic

$a \equiv b \pmod{m}$ iff $m \mid (a - b)$ iff $a \bmod m = b \bmod m$. Residues $\mathbb{Z}_m = \{0, \dots, m-1\}$ form a clock.

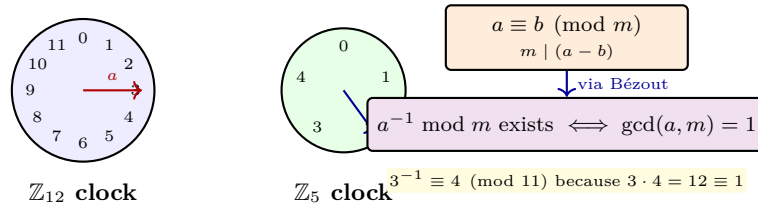


Figure 8.3: Modular arithmetic as clock arithmetic. Invertibility depends on coprimality.

Arithmetic mod m : $(a + b) \bmod m$, $(a \cdot b) \bmod m$ stay in $\mathbb{Z}_m$. Division by a means multiply by $a^{-1} \bmod m$, which exists iff $\gcd(a, m) = 1$.

Theorem 8.3 (Fermat's little theorem). If p prime and $p \nmid a$ then $a^{p-1} \equiv 1 \pmod{p}$, hence $a^p \equiv a \pmod{p}$ always.

Sketch. Non-zero residues permuted by multiplication by a ; product $1 \cdot 2 \cdots (p-1) \equiv a^{p-1} \cdot 1 \cdot 2 \cdots (p-1)$. $\square$

Chinese remainder theorem (CRT): for coprime m_1, m_2 , the pair $(a \bmod m_1, a \bmod m_2)$ uniquely determines $a \bmod m_1 m_2$; it stitches clocks together.

8.4 RSA Intuition

RSA turns Fermat/Euler into a trapdoor. Pick large primes p, q , $n = pq$, $\phi(n) = (p-1)(q-1)$. Choose e coprime to $\phi(n)$ (public exponent), compute $d \equiv e^{-1} \pmod{\phi(n)}$ (private, via extended Euclid). Public key (n, e) , private d .

Encrypt $m \in [0, n)$: $c = m^e \bmod n$. Decrypt: $m = c^d \bmod n$ because $m^{ed} = m^{1+k\phi(n)} \equiv m$ by Euler ($a^{\phi(n)} \equiv 1$ when coprime; CRT handles rest).

Security rests on hardness of factoring n to get $\phi(n)$; correctness on modular inverse and FLT/Euler.

```

1  # Toy RSA (tiny primes -- never use like this in production!)
2  p, q = 61, 53
3  n = p*q; phi = (p-1)*(q-1)
4  e = 17
5  # d = e^{-1} mod phi
6  def egcd(a,b):
7      if b==0: return (a,1,0)
8      g,x1,y1=egcd(b,a%b)
9      return (g,y1,x1-(a//b)*y1)
10 d = egcd(e,phi)[1] % phi
11 m = 42
12 c = pow(m,e,n)
13 m2 = pow(c,d,n)
14 print(f"n={n} phi={phi} e={e} d={d} c={c} decrypted={m2}")
15 assert m==m2

```

Remark 8.1 (Why not just m^e ?). Modular exponentiation via repeated squaring keeps numbers $< n^2$; m^e without mod would be astronomically huge ($\approx 2^{2048}$ bits for real RSA-2048).

Theorem 8.4 (Euler's theorem). If $\gcd(a, m) = 1$ then $a^{\phi(m)} \equiv 1 \pmod{m}$, where ϕ is Euler's totient. FLT is the prime case $\phi(p) = p - 1$.

Example 8.1 (Modular exponentiation). $2^{10} \bmod 1000$ computed by squaring: $2^2 = 4$, $2^4 = 16$, $2^8 = 256$, $2^{10} = 256 \cdot 4 = 1024 \equiv 24$; no need for huge intermediates.

Divisibility is the discrete analogue of inequality: $a \mid b$ means b is a multiple of a . Division algorithm is Euclidean division: every integer sits uniquely in a block of size d with remainder r . Primes are the atoms of multiplication, uniquely factoring every integer beyond 1. Euclid's algorithm is the oldest non-trivial algorithm, its logarithmic steps still optimal. Bézout's identity shows $\gcd(a, b)$ is not just a divisor but a linear combination, enabling inverses. Modular arithmetic is clock arithmetic: numbers wrap around after m , forming the ring $\mathbb{Z}_m$. Invertible residues mod m are exactly those coprime to m , counted by Euler's totient $\phi(m)$. Fermat's little theorem gives a fast exponent reduction, powering primality tests and RSA. Chinese remainder theorem stitches independent clocks into one, enabling parallel modular computations. RSA's security rests on factoring being hard while multiplying primes is easy, a computational asymmetry. Divisibility is the discrete analogue of inequality: $a \mid b$ means b is a multiple of a . Division algorithm is Euclidean division: every integer sits uniquely in a block of size d with remainder r . Primes are the atoms of multiplication, uniquely factoring every integer beyond 1. Euclid's algorithm is the oldest non-trivial algorithm, its logarithmic steps still optimal. Bézout's identity shows $\gcd(a, b)$ is not just a divisor but a linear combination, enabling inverses. Modular arithmetic is clock arithmetic: numbers wrap around after m , forming the ring $\mathbb{Z}_m$. Invertible residues mod m are exactly those coprime to m , counted by Euler's totient $\phi(m)$. Fermat's little theorem gives a fast exponent reduction, powering primality tests and RSA. Chinese remainder theorem stitches independent clocks into one, enabling parallel modular computations. RSA's security rests on factoring being hard while multiplying primes is easy, a computational asymmetry. Divisibility is the discrete analogue of inequality: $a \mid b$ means b is a multiple of a . Division algorithm is Euclidean division: every integer sits uniquely in a block of size d with remainder r . Primes are the atoms of multiplication, uniquely factoring every integer beyond 1. Euclid's algorithm is the oldest non-trivial algorithm, its logarithmic steps still optimal. Bézout's identity shows $\gcd(a, b)$ is not just a divisor but a linear combination, enabling inverses. Modular arithmetic is clock arithmetic: numbers wrap around after m , forming the ring $\mathbb{Z}_m$. Invertible residues mod m are exactly those coprime to m , counted by Euler's totient $\phi(m)$. Fermat's little theorem gives a fast exponent reduction, powering primality tests and RSA. Chinese remainder theorem stitches independent clocks into one, enabling parallel modular computations. RSA's security rests on factoring being hard while multiplying primes is easy, a computational asymmetry.

8.5 Worked Example: A Full Modular Calculation

Solve $6x \equiv 9 \pmod{15}$. $\gcd(6, 15) = 3$ divides 9, so solutions exist ($3 \mid 9$). Divide by 3: $2x \equiv 3 \pmod{5}$. Now $\gcd(2, 5) = 1$, invert $2^{-1} \equiv 3 \pmod{5}$ because $2 \cdot 3 = 6 \equiv 1$. So $x \equiv 3 \cdot 3 = 9 \equiv 4 \pmod{5}$. Hence $x \equiv 4, 9, 14 \pmod{15}$ (three solutions, one per coset). If we tried $6x \equiv 7 \pmod{15}$, $3 \nmid 7$ gives no solution — the congruence is inconsistent, like $0 \equiv 1$.

Compute $7^{100} \bmod 15$: $\phi(15) = 8$, 7 coprime to 15, so $7^8 \equiv 1$, $7^{100} = (7^8)^{12} \cdot 7^4 \equiv 7^4$. $7^2 = 49 \equiv 4$, $7^4 \equiv 4^2 = 16 \equiv 1$, so $7^{100} \equiv 1 \pmod{15}$.

8.6 Chapter Notes

Euclid (~ 300 BCE) gave the first non-trivial algorithm; Gauss systematised congruences (1801). Fermat, Euler, and CRT underpin modern cryptography (Rivest–Shamir–Adleman, 1977). The hardness of factoring vs. primality testing (AKS, 2002) separates feasible from believed-hard.

8.7 Exercises

- E8.1 **Divisibility.** Prove if $a \mid b$ and $b \mid c$ then $a \mid c$; give a counterexample that $a \mid bc$ does not imply $a \mid b$ or $a \mid c$.
- E8.2 **Euclid.** Compute $\gcd(252, 198)$ via Euclid, then find x, y with $252x + 198y = \gcd$ using extended Euclid. Compute $\text{lcm}(252, 198)$.
- E8.3 **Mod inverses.** Find $5^{-1} \bmod 11$ and $6^{-1} \bmod 11$ or explain why none exists for a case. Solve $3x \equiv 7 \pmod{11}$.
- E8.4 **Fermat.** Use FLT to compute $2^{100} \bmod 11$ and $3^{16} \bmod 17$ without a calculator. Check with Python `pow`.
- E8.5 **RSA toy.** With $p = 5, q = 11, e = 3$ compute $n, \phi(n), d$, encrypt $m = 4$, decrypt and verify. Why must e be coprime to $\phi(n)$?